

Course Notes for Applied Engineering Optimization

Jeremy Watson, Joe Chen and Le Yang
Department of Electrical and Computer Engineering
University of Canterbury

February 2026

The textbook for this course is freely downloadable from:

<https://mdobook.github.io/>

Chapter 1

Introduction to Optimization

Optimization mathematically refers to the process of finding the best possible solution for a set of variables that can be controlled, often subject to constraints on those variables. It is applicable to any area where a decision needs to be made, as people always seek *improvement*. The field of optimization has also become central in the era of big data and modern applications such as power network optimization, image processing, and machine learning.

Most practical optimization problems are too complicated to be solved analytically (i.e., in general, we cannot obtain closed-form solutions), so we have to resort to numerical computation in order to find a solution.

1.1 A Short History and the Future of Optimization

Numerical optimization emerged in the field of *Operations Research* that concerns problems like product-pricing, distribution network setup, and the classic Travelling Salesman Problem (TSP), which addresses the question: given a list of cities and the distances between each pair of cities, what is the shortest possible route that visits each city exactly once and returns to the original city?

The concept of optimizing things itself is much much older, best documented in ancient Greece, but perhaps the very first humans would have had the idea of optimization on their minds. For an overview of the application of numerical optimization to engineering design, please refer to [Chapter 1.1](#) of the textbook.

1.2 Numerical Optimization Process

A typical numerical optimization process consists of the following steps (see also [Fig. 1.4](#) of the textbook):

1. **Problem Description:** This step requires *writing down* a description of the problem, including system specifications, goals, requirements and constraints. Market research, similar design survey, multidisciplinary collaborations and application of your expertise/background knowledge may be needed.
2. **Information Gathering:** This step gathers and organizes, through conducting more research, data and information related to problem description. The better you understand the problem, the more likely you will formulate a sound optimization

problem (to avoid “*finding a correct solution to the wrong problem*”) and recognize possible issues in the obtained solutions.

3. **Problem Formulation:** This step *translates* the design goal/requirements and specifications into a mathematical problem that can be solved by a numerical optimization algorithm. We are going to elaborate more on this step in Section 1.3.
4. **Problem Solving:** This step requires selecting and possibly developing a numerical optimization algorithm to find the solution to the formulated optimization problem. The algorithm usually outputs a solution that satisfies particular optimal conditions, and its execution, once started, normally does not need intervention from the designer.
5. **Solution Assessment:** This step requires analyzing the *quality* of the obtained solution to determine whether the solution is valid, and satisfies the design goals and requirements. Re-formulating and re-solving the optimization problem by e.g., changing the objective function or design variables, adding/removing constraints, and selecting or designing a different optimization algorithm.
6. **Post-optimality Studies:** This step is performed to better *interpret* the results and possibly simplify or gain more insights into the problem through carrying out e.g., sensitivity analysis that aims at identifying which design variables are more influential or which constraints drive the design.

1.3 Optimization Problem Formulation

In general, formulating an optimization problem requires specifying the **design variables**, **objective function(s)** and **constraints**.

1.3.1 Design variables

The **design variables** are variables whose values correspond to different optimization solutions. They are the inputs to the **objective function**, which can be evaluated or simulated to quantify the *quality* of a solution. Mathematically, we collect the design variables into a $n \times 1$ column vector $\mathbf{x}$, which is equal to

$$\mathbf{x} = [x_1, x_2, \dots, x_n]^T \quad (1.1)$$

where n is the dimensionality of the optimization problem.

The design variables should be independent to one another. The presence of linearly dependent design variables can lead to an infinite number of solutions to the optimization problem.

The choice of design variables may not be unique (see [Example 1.1](#) in the textbook for an example). Depending on the applications, the design variables could be *continuous* or *discrete*. If an optimization problem has at least one discrete design variables, it becomes a discrete optimization problem. If a design variable x_i , $i = 1, 2, \dots, n$, is continuous, we normally allow it to vary within a range $[\underline{x}_i, \bar{x}_i]$ such that x_i must satisfy

$$\underline{x}_i \leq x_i \leq \bar{x}_i. \quad (1.2)$$

The inclusion of bound constraints should not be artificial. A small range for a design variable (e.g., $\bar{x}_i - \underline{x}_i$ is small) means a small search space for x_i , which may limit the solution quality.

Transformations of design variables are useful in some scenarios to eliminate bound constraints. For instance, if x_i must be non-negative (i.e., $\underline{x}_i = 0$ and $\bar{x}_i = +\infty$), we may replace it with $\exp(x_i)$ or the softplus activation $\log(1 + \exp(x_i))$ so that the bound constraints become $x_i \in \mathcal{R}$, i.e., x_i now becomes unbounded.

For a complex practical problem, optimization practitioners should start with a small set of design variables and then move to higher-fidelity modeling with more design variables.

1.3.2 Objective functions

The **objective function** $f(\mathbf{x})$ takes the design variable vector $\mathbf{x}$ as input and outputs a value that quantifies the quality of $\mathbf{x}$. Without loss of generality, in this note, we assume that the smaller the objective function value is, the better the quality of $\mathbf{x}$ would be. That is, the goal of the optimization problem is to minimize the objective function $f(\mathbf{x})$. Notice that this does not prevent us from maximizing a function. In fact, we can simply minimize the negative of that function to achieve the desired purpose (see Fig. 1.8 in the textbook for an illustration).

$f(\mathbf{x})$ can have algebraic closed-form expressions (e.g., $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T\mathbf{x}$). It can also be a system simulator or even a physical system composed of multiple coupled modules. In the latter case, evaluating the objective function may be time-consuming and costly. In this case, surrogate modeling (see Chapter 10 of the textbook for an introduction) may be used to replace the true objective function.

The choice of the objective function is probably the core of a successful optimization practice (in fact, sophisticated objective function design that includes various regularizers is essential for the success of some deep neural network training as well). A poor choice of the objective function may not represent the true intent of the practitioner and lead to a useless solution. Experimenting with different objective functions is a common process, as they could provide more insights into the problem itself.

Visualization of the objective function sometimes is very useful for e.g., understanding the convergence behaviours of the optimization algorithms. One possibility is to use the contour of the objective function, which are lines of the constant objective function values (see Fig. 1.10 in the textbook for an example). In the textbook, by convention, darker lines denote lower objective function values (better solutions), while lighter lines represent higher objective function values (poorer solutions). Regions with dense lines indicate rapid changes in the objective function values.

1.3.3 Constraints

Constraints are functions of the design variables that enforce restrictions in certain ways. If a solution satisfies all the constraints of a problem, it is called a *feasible solution*. The set of all feasible solutions is termed a *feasible region*. The constraints may come from practical considerations such as the limit of transmission power and the functional relationship between the design variables.

If the constraint function is equal to a fixed value, the corresponding constraint is an *equality constraint*. An equality constraint can be generically written as $h(\mathbf{x}) = 0$. When

the constraint function is required to be smaller than or equal to a particular value, the associated constraint is called an *inequality constraint*, which is denoted as $g(\mathbf{x}) \leq 0$ ¹.

An equality constraint $h(\mathbf{x}) = 0$ can be replaced by the combination of two inequality constraints $h(\mathbf{x}) \geq 0$ and $h(\mathbf{x}) \leq 0$.

An inequality constraint $g(\mathbf{x}) \leq 0$ is *active* for a certain solution if it attains equality. It becomes *inactive* if for a certain solution, it satisfies $g(\mathbf{x}) < 0$. An important property for inequality constraints is that when an inequality constraint is known to be inactive at the optimal solution, it can be safely removed from the formulated optimization problem without affecting the quality of the solutions (see Fig. 1.12 for an illustration on the active and inactive inequality constraints).

Over-constraining the problem is one common issue in optimization problem formulations. This could lead to the increased difficulty of the problem or even make the problem infeasible (i.e., the feasible region is empty). In this case, constraints need to be removed or relaxed. Mathematically, the number of equality constraints cannot be greater than the dimensionality of the design variable vector $\mathbf{x}$, which is n . There is no limit on the number of inequality constraints. Reducing the number of constraints would lead to increased feasible region and as a result, reduced/the same minimum objective function value (i.e., improved/the same solution quality). Adding more constraints gives the opposite effect.

1.4 Optimization Problem Statement

An optimization problem can be stated as follows:

Vary the design variables within their bounds to minimize the value of the objective function while satisfying all the equality and inequality constraints.

Mathematically, an optimization problem can be expressed as

$$\begin{aligned} & \min f(\mathbf{x}) \\ & \text{subject to } \underline{x}_i \leq x_i \leq \bar{x}_i, i = 1, 2, \dots, n \\ & \quad g_j(\mathbf{x}) \leq 0, j = 1, 2, \dots, n_g \\ & \quad h_l(\mathbf{x}) = 0, l = 1, 2, \dots, n_h \end{aligned} \tag{1.3}$$

- n_g denotes the number of inequality constraints while $n_h \leq n$ represents the number of equality constraints.
- It is required that the objective function $f(\mathbf{x})$ and constraints, $g_j(\mathbf{x})$ and $h_l(\mathbf{x})$, be *computable*.
- The bounds $\underline{x}_i$ and $\bar{x}_i$ should come from the physical constraints on the design variables.

When applying gradient-based optimization algorithms, it is usually quite useful to plot/visualize the objective function values and the gradients across iterations to verify the validity of the gradient evaluation and investigate the algorithm behavior.

¹A strict inequality $g(\mathbf{x}) < 0$ is never used in optimization. If the optimal solution $\mathbf{x}^*$ makes the inequality constraint active (i.e., $g(\mathbf{x}^*) = 0$), the use of a strict inequality will render the problem ill-defined.

1.5 Optimization Problem Classification

Understanding the attributes of an optimization problem is essential for choosing an appropriate optimization algorithm, because no algorithm is efficient for all types of problems.

We shall adopt a classification based on the characteristics of the objective and constraint functions in the optimization problem. This course, to limit the scope, focuses mostly on single objective optimization problems with continuous design variables. For an introduction to multi-objective optimization, please refer to [Chapter 9](#) of the textbook. If some of the design variables are discrete, we have a *mixed* problem (see [Chapter 9](#) of the textbook for an introduction to discrete optimization).

Smoothness: A function is called C^0 continuous, if its function value varies continuously. It is called C^1 continuous, if its first-order derivative varies continuously. $|x|$ is a C^0 continuous but not C^1 continuous function.

A function is smooth when the derivatives of all orders vary continuously, a rather strong condition and sometimes hard to verify.

Linearity: If the objective function and constraints are linear, the optimization problem is termed as a linear optimization/programming² problem.

If the optimization problem has a quadratic objective function and linear constraints, it is called a quadratic optimization problem.

In general, practical optimization problems are nonlinear. But many optimization algorithms take advantage of the techniques used for solving linear and quadratic problems.

Multimodality: A function can be unimodal or multimodal. The difference is that a multimodal function has more than one minimum (they are called *local minima*, as they are minima within small neighborhoods), while a unimodal function has only one minimum (i.e., the *global minimum*). In many cases, we do not have the knowledge on whether a function is unimodal or not. But proving by finding two local minima to show that a function is multimodal is much easier.

Finding the global minimum is always desired. For an optimization problem with a multimodal objective function, if we start with different initial guesses for the solution and our optimization algorithm always converges to the same solution, we then have some confidence that we may have located the global minimum (but still there is no guarantee).

Among the unimodal functions, convex functions are of special interests (in fact, many convergence analysis of the optimization algorithms assume convexity). For a function to be convex, the line segment connecting any two points in the function will be above the function and never intercept it. Convex optimization problems have a convex objective function with a feasible region which is also a convex set. We have specialized algorithms that are efficient in solving for the global minimum of the convex optimization problem. See [Fig. 1.20](#) in the textbook for an illustration of the unimodal, convex and multimodal functions.

²The word “programming” comes from the field of Operations Research.

1.6 Optimization Algorithm Overview

The numerical optimization algorithms that will be discussed in detail in this course are iterative in the sense that the algorithm generally starts with an initial solution and gradually improves it through searching for solutions providing smaller objective function values (i.e., solutions of better quality). Depending on what type of information is utilized to update a solution, the optimization algorithms can be categorized into *gradient-based*, *gradient-free*, and *curvature-based* methods.

- **Gradient-based** algorithms use gradients of the objective function and constraints with respect to the design variables in updating the solution. Gradients provide information regarding the *local* rate of change in the function values, which can be explored to guide the update of the solution and speed up the convergence.
- **Gradient-free** methods explore the values of the objective function and constraints of the optimization problem only. Sometimes, they are called zeroth-order techniques as well. These methods are relatively easy to set up and more robust because they only require that the objective function and constraints are numerically computable (no smoothness and modality of the functions are assumed). Some widely used gradient-free methods are heuristic and nature-inspired.
- **Curvature-based** methods explore the second-order information (i.e., the curvature) to better describe the local function behaviors.

Note that the gradient information may be found by finding the closed-form expression of the first-order derivative of the function. It can also be determined numerically using finite difference approximations. Similarly, the curvature information may be obtained numerically using the gradient information as well, as in some Newton-type second-order optimization algorithms.

Gradients can be used to establish more rigorous criteria for “local” optimality (i.e., to decide whether we have reached a local minimum and need to terminate the iteration). In contrast, gradient-free methods have difficulties in verifying whether a solution found is a local minimum or not. But they are still quite useful in some applications, as they may find solutions that produce smaller objective functions and have better quality, and they can still be applied even if the objective function is not differentiable.

An advantage of gradient-based methods is that their complexity in terms of objective function evaluations scales nicely with the dimensionality of the problem. On the other hand, gradient-free methods have in general, an exponentially increasing number of objective function evaluations as the problem dimensionality becomes larger.

Another point worthwhile of mentioning is that gradient-based and curvature-based methods normally perform *local* search as they utilize local information only. But this does not prevent them from traversing large portions of the design variable space. Some gradient-free methods can perform global search through starting with multiple initial solutions and carrying out parallel searching. These strategies can be utilised in gradient-based methods, e.g., we can start the gradient-based method with multiple initial solutions to improve its performance and robustness.

In this course, we are going to discuss gradient-based algorithms (including numerical evaluation of gradients) for unconstrained optimization problems (by Le Yang) and constrained optimization problems (by Jeremy Watson), as well as gradient-free methods.

Chapter 2

Mathematics Background: Part I

In these notes, scalars are represented using regular letters (e.g., a and A). Boldface lowercase letters such as $\mathbf{a}$ are used to denote vectors. Matrices are denoted by boldface capital letters like $\mathbf{A}$.

2.1 Basic Vector and Matrix Manipulations

Let $\mathbf{x}$ and $\mathbf{A}$ denote a $n \times 1$ column vector and a $m \times n$ matrix. They can be expressed as

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}, \quad (2.1)$$

where x_i is the i -th element in $\mathbf{x}$, and

$$\mathbf{A} = \begin{bmatrix} A_{11} & A_{12} & \cdots & A_{1n} \\ A_{21} & A_{22} & \cdots & A_{2n} \\ \vdots & \vdots & \cdots & \vdots \\ A_{m1} & A_{m2} & \cdots & A_{mn} \end{bmatrix}, \quad (2.2)$$

where A_{ij} is the element in the i -th row and j -th column of $\mathbf{A}$, $i = 1, 2, \dots, m$ and $j = 1, 2, \dots, n$.

In general, it is convenient to write the matrix $\mathbf{A}$ in the *partitioned form*. For this purpose, define $\mathbf{a}_j$ as the collection of elements in the j -th column of $\mathbf{A}$ in (2.2) in a $m \times 1$ column vector. That is,

$$\mathbf{a}_j = \begin{bmatrix} A_{1j} \\ A_{2j} \\ \vdots \\ A_{mj} \end{bmatrix}. \quad (2.3)$$

With $\mathbf{a}_j$ in (2.3), $j = 1, 2, \dots, n$, $\mathbf{A}$ can be re-written as

$$\mathbf{A} = [\mathbf{a}_1, \mathbf{a}_2, \dots, \mathbf{a}_n]. \quad (2.4)$$

As a result, we have that the matrix-vector product $\mathbf{Ax}$ can be computed using

$$\mathbf{Ax} = [\mathbf{a}_1, \mathbf{a}_2, \dots, \mathbf{a}_n] \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \sum_{i=1}^n \mathbf{a}_i x_i. \quad (2.5)$$

Applying the *transpose* operations $\mathbf{x}^T$ and $\mathbf{A}^T$ yields a $1 \times n$ row vector given by

$$\mathbf{x}^T = [x_1, x_2, \dots, x_n], \quad (2.6)$$

and $n \times m$ matrix given by

$$\mathbf{A}^T = \begin{bmatrix} \mathbf{a}_1^T \\ \mathbf{a}_2^T \\ \vdots \\ \mathbf{a}_n^T \end{bmatrix} = \begin{bmatrix} A_{11} & A_{21} & \cdots & A_{m1} \\ A_{12} & A_{22} & \cdots & A_{m2} \\ \vdots & \vdots & \cdots & \vdots \\ A_{1n} & A_{2n} & \cdots & A_{mn} \end{bmatrix}. \quad (2.7)$$

The first equality in (2.7) was obtained through taking the transpose on both sides of (2.4).

The *inner product* of two vectors $\mathbf{x}$ and $\mathbf{y}$ is

$$\mathbf{x}^T \mathbf{y} = \mathbf{y}^T \mathbf{x} = \sum_{i=1}^n x_i y_i, \quad (2.8)$$

where $\mathbf{y} = [y_1, y_2, \dots, y_n]^T$.

The *outer product* of two vectors $\mathbf{x}$ and $\mathbf{y}$ is

$$\mathbf{xy}^T = \sum_{i=1}^n y_i \mathbf{x} = \begin{bmatrix} x_1 y_1 & x_1 y_2 & \cdots & x_1 y_n \\ x_2 y_1 & x_2 y_2 & \cdots & x_2 y_n \\ \vdots & \vdots & \cdots & \vdots \\ x_n y_1 & x_n y_2 & \cdots & x_n y_n \end{bmatrix}. \quad (2.9)$$

2.2 Vector and Matrix Norms

Norms generalize the operation of finding the absolute value of a scalar (i.e., $|x|$). They provide an idea of the magnitude of the elements in vectors and matrices. Some commonly used vector and matrix norms are presented as follows.

Let $\mathbf{x} = [x_1, x_2, \dots, x_n]^T$ be a $n \times 1$ column vector. The 2-norm of $\mathbf{x}$, also called the Euclidean norm, characterizes the Euclidean length of $\mathbf{x}$, which is defined as

$$\|\mathbf{x}\|_2 = (x_1^2 + x_2^2 + \dots + x_n^2)^{\frac{1}{2}} = \left(\sum_{i=1}^n x_i^2 \right)^{\frac{1}{2}}. \quad (2.10)$$

It can also be verified that the square of the 2-norm of $\mathbf{x}$ is

$$\|\mathbf{x}\|_2^2 = (x_1^2 + x_2^2 + \dots + x_n^2) = \left(\sum_{i=1}^n x_i^2 \right) = \mathbf{x}^T \mathbf{x}. \quad (2.11)$$

The 2-norm actually is a member of the class of p -norms. Specifically, the p -norm of $\mathbf{x}$ is equal to

$$\|\mathbf{x}\|_p = (|x_1|^p + |x_2|^p + \dots + |x_n|^p)^{\frac{1}{p}} = \left(\sum_{i=1}^n |x_i|^p \right)^{\frac{1}{p}}. \quad (2.12)$$

When $p = 1$, we have the 1-norm given by

$$\|\mathbf{x}\|_1 = |x_1| + |x_2| + \dots + |x_n| = \left(\sum_{i=1}^n |x_i| \right). \quad (2.13)$$

When p approaches $+\infty$, the element with the largest magnitude in $\mathbf{x}$ would dominate the sum in (2.12). In this case, the infinity norm is

$$\begin{aligned} \|\mathbf{x}\|_\infty &= \lim_{p \rightarrow +\infty} \left(|x_{\max}|^p \sum_{i=1}^n \left| \frac{x_i}{x_{\max}} \right|^p \right)^{\frac{1}{p}} \\ &= \lim_{p \rightarrow +\infty} \left(|x_{\max}|^p \right)^{\frac{1}{p}} \\ &= |x_{\max}|, \end{aligned} \quad (2.14)$$

where $|x_{\max}| = \max_i |x_i|$.

The contours of the vector norms shown above are visualized in [Fig. A.8](#) on page 549 of the textbook.

Let $\mathbf{A}$ be a $m \times n$ matrix. The Frobenius norm of $\mathbf{A}$ is defined as the square root of the sum of the elements in $\mathbf{A}$ squared (compare with (2.10)), which is given by

$$\|\mathbf{A}\|_F = \left(\sum_{i=1}^m \sum_{j=1}^n A_{ij}^2 \right)^{\frac{1}{2}}. \quad (2.15)$$

It can be shown ([Exercise 3.1](#)) that

$$\|\mathbf{A}\|_F^2 = \text{tr}(\mathbf{A}^T \mathbf{A}). \quad (2.16)$$

The Frobenius norm can be weighted by a matrix $\mathbf{W}$ as follows

$$\|\mathbf{A}\|_W = \|\mathbf{W}^{\frac{1}{2}} \mathbf{A} \mathbf{W}^{\frac{1}{2}}\|_F. \quad (2.17)$$

2.3 Quadratic Forms

The product $\mathbf{x}^T \mathbf{B} \mathbf{x}$ is called the *quadratic form*. It is equal to, after applying $\mathbf{B} = [\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_n]$ (see (2.4)), and the results in (2.8) and (2.5),

$$\begin{aligned} \mathbf{x}^T \mathbf{B} \mathbf{x} &= \mathbf{x}^T \sum_{j=1}^n \mathbf{b}_j x_j \\ &= \sum_{j=1}^n (\mathbf{x}^T \mathbf{b}_j) x_j \\ &= \sum_{j=1}^n x_j \sum_{i=1}^n B_{ij} x_i \\ &= \sum_{i=1}^n \sum_{j=1}^n B_{ij} x_i x_j. \end{aligned} \quad (2.18)$$

Another way to compute the quadratic form is to use the matrix trace. Given a symmetric matrix, its trace is denoted as $\text{tr}(\mathbf{B})$, which is equal to

$$\text{tr}(\mathbf{B}) = \sum_{i=1}^n B_{ii}. \quad (2.19)$$

In other words, the trace of a matrix is just the sum of its diagonal elements. An important property of the matrix trace is

$$\mathbf{x}^T \mathbf{y} = \text{tr}(\mathbf{x}^T \mathbf{y}) = \text{tr}(\mathbf{y} \mathbf{x}^T). \quad (2.20)$$

Thus, noting that in (2.18), $\mathbf{B} \mathbf{x}$ is a column vector, we have

$$\mathbf{x}^T \mathbf{B} \mathbf{x} = \mathbf{x}^T (\mathbf{B} \mathbf{x}) = \text{tr}(\mathbf{x}^T (\mathbf{B} \mathbf{x})) = \text{tr}((\mathbf{B} \mathbf{x}) \mathbf{x}^T) = \text{tr}(\mathbf{B} (\mathbf{x} \mathbf{x}^T)). \quad (2.21)$$

The matrix $\mathbf{B}$ is called *symmetric* if $\mathbf{B} = \mathbf{B}^T$. A symmetric matrix must be a square matrix (i.e., the numbers of rows and columns are equal).

The symmetric matrix $\mathbf{B}$ is *positive definite*, if for *any* non-zero vector $\mathbf{x}$, we have

$$\mathbf{x}^T \mathbf{B} \mathbf{x} > 0. \quad (2.22)$$

It becomes a *positive semidefinite* matrix, if for *any* non-zero vector $\mathbf{x}$, we have

$$\mathbf{x}^T \mathbf{B} \mathbf{x} \geq 0. \quad (2.23)$$

A necessary but not sufficient condition for a matrix $\mathbf{B}$ to be positive definite is that its diagonal elements are all positive (i.e., $B_{ii} > 0$) and its off-diagonal elements satisfy $B_{ij} < \sqrt{B_{ii} B_{jj}}$.

An equivalent condition to (2.23) is that the eigenvalues of $\mathbf{B}$ are non-negative. Similarly, an equivalent condition to (2.22) is that the eigenvalues of $\mathbf{B}$ are all positive.

There are two commonly used numerical approaches to determine whether a symmetric matrix $\mathbf{B}$ is positive definite or not. The first method is to examine all the leading

principal minors of $\mathbf{B}$. A leading principal minor is the determinant of a leading principal submatrix, where the leading principal submatrix of order k of $\mathbf{B}$ is obtained by removing the last $n - k$ rows and $n - k$ columns of $\mathbf{B}$. For a graphic illustration of the leading principal matrix, see [Fig.A.9 in the textbook on page 551](#). Therefore, we can start with computing the leading principal minor of the leading principal submatrix of order 1, which is exactly equal to B_{11} , and continue until we reach order n to confirm the positive definiteness of $\mathbf{B}$. If for any order, the corresponding leading principal minor is not positive, we can immediately declare that the matrix is not positive definite.

The other approach is to perform the Cholesky decomposition of $\mathbf{B}$. The Cholesky decomposition represents the positive definite matrix $\mathbf{B}$ as the product of a lower triangle matrix $\mathbf{L}$ and its transpose (i.e., $\mathbf{B} = \mathbf{L}\mathbf{L}^T$). The Cholesky decomposition is an iterative process, which would terminate due to e.g., taking the square root of a negative number or dividing by zero if the matrix is not positive definite. Therefore, if we cannot find the Cholesky decomposition of $\mathbf{B}$, it cannot be positive definite.

2.4 Useful Derivatives

2.4.1 Gradient

Consider a scalar multivariate function $f(\mathbf{x})$, whose input is a $n \times 1$ column vector $\mathbf{x}$. The gradient of $f(\mathbf{x})$, denoted by $\nabla f(\mathbf{x})$ is also a column vector, which is defined to be the vector derivative of $f(\mathbf{x})$ and it is equal to

$$\nabla f(\mathbf{x}) = \frac{\partial f(\mathbf{x})}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial f(\mathbf{x})}{\partial x_1} \\ \frac{\partial f(\mathbf{x})}{\partial x_2} \\ \vdots \\ \frac{\partial f(\mathbf{x})}{\partial x_n} \end{bmatrix}. \quad (2.24)$$

The i -th element in $\nabla f(\mathbf{x})$, $\frac{\partial f(\mathbf{x})}{\partial x_i}$, gives the *rate of change* of the function $f(\mathbf{x})$ along the i -th coordinate axis.

Let $\mathbf{p}$ be a direction vector such that $\|\mathbf{p}\|_2 = 1$. Then, for a sufficiently small step size μ , $\mu \nabla f(\mathbf{x})^T \mathbf{p}$ is the amount of change in the function value along the direction $\mathbf{p}$.

2.4.2 Hessian

The Hessian of $f(\mathbf{x})$ with respect to $\mathbf{x}$, which is the vector extension of the second-order derivative of a scalar univariate function, is defined as

$$\mathbf{H}_f(\mathbf{x}) = \frac{\partial f(\mathbf{x})}{\partial \mathbf{x} \partial \mathbf{x}^T} = \begin{bmatrix} \frac{\partial^2 f(\mathbf{x})}{\partial x_1^2} & \frac{\partial^2 f(\mathbf{x})}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f(\mathbf{x})}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f(\mathbf{x})}{\partial x_2 \partial x_1} & \frac{\partial^2 f(\mathbf{x})}{\partial x_2^2} & \cdots & \frac{\partial^2 f(\mathbf{x})}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \cdots & \vdots \\ \frac{\partial^2 f(\mathbf{x})}{\partial x_n \partial x_1} & \frac{\partial^2 f(\mathbf{x})}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f(\mathbf{x})}{\partial x_n^2} \end{bmatrix}. \quad (2.25)$$

It can be seen by comparing with (2.24) that the i -th row of the Hessian is in fact the gradient (after transpose) of the partial derivative $\frac{\partial f(\mathbf{x})}{\partial x_i}$.

Again, let $\mathbf{p}$ be a direction vector such that $\|\mathbf{p}\|_2 = 1$. Then, for a sufficiently small step size μ , $\mu \mathbf{H}_f(\mathbf{x})\mathbf{p}$ is a column vector whose i -th element is the amount of change in the partial derivative $\frac{\partial f(\mathbf{x})}{\partial x_i}$ along the direction $\mathbf{p}$.

What about $\mu^2 \mathbf{p}^T \mathbf{H}_f(\mathbf{x}) \mathbf{p}$? It is basically the projection of $\mu \mathbf{H}_f(\mathbf{x})\mathbf{p}$ along $\mathbf{p}$. It is called the *curvature* of the function $f(\mathbf{x})$ around $\mathbf{x}$ along $\mathbf{p}$. It models the change of the function value $f(\mathbf{x})$ along the direction $\mathbf{p}$ due to the variation in the gradient.

If a direction vector satisfies

$$\mathbf{H}_f(\mathbf{x})\mathbf{p} = \kappa \mathbf{p}, \quad (2.26)$$

it is called a *principal curvature direction*. In this case, the associated curvature along $\mathbf{p}$ becomes $\mu^2 \mathbf{p}^T \mathbf{H}_f(\mathbf{x}) \mathbf{p} = \mu^2 \kappa \mathbf{p}^T \mathbf{p} = \kappa \mu^2$, which is proportional to κ .

Superimposing the principal curvature directions on the function contours (see an example in Fig. 4.8 in the textbook) illustrates how the function curvature varies as a function of the direction vector $\mathbf{p}$.

The gradient in (2.24) and Hessian in (2.25) capture the first-order and second-order variations of the function $f(\mathbf{x})$ around a point $\mathbf{x}$.

2.4.3 Vector derivatives

Consider the case of vector inner product (i.e., $f(\mathbf{x}) = \mathbf{a}^T \mathbf{x}$, where $\mathbf{a}$ is a $n \times 1$ column vector, and see (2.8)). It can be shown (Exercise 3.2 (a)) that by the product rule of differentiation in calculus, we have

$$\frac{\partial \mathbf{a}^T \mathbf{x}}{\partial \mathbf{x}} = \mathbf{a}. \quad (2.27)$$

Next, we consider the case of quadratic form (i.e., $f(\mathbf{x}) = \mathbf{x}^T \mathbf{A} \mathbf{x}$, where $\mathbf{A}$ is a $n \times n$ matrix and see (2.18)). It can be shown (Exercise 3.2 (b)) that

$$\frac{\partial \mathbf{x}^T \mathbf{A} \mathbf{x}}{\partial \mathbf{x}} = \mathbf{A}^T \mathbf{x} + \mathbf{A} \mathbf{x}. \quad (2.28)$$

If $\mathbf{A}$ is symmetric, the above result becomes

$$\frac{\partial \mathbf{x}^T \mathbf{A} \mathbf{x}}{\partial \mathbf{x}} = 2\mathbf{A} \mathbf{x}. \quad (2.29)$$

2.5 Taylor-Series Expansion

2.5.1 Univariate case

In calculus, we have learned that for a scalar univariate function, $f(x)$, the line tangent to $f(x)$ at a particular point $(x_0, f(x_0))$ is

$$y - f(x_0) = f'(x_0)(x - x_0), \quad (2.30)$$

where $f'(x_0)$ is the slope equal to the first-order derivative of $f(x)$ evaluated at $x = x_0$. The linearization of $f(x)$ at $x = x_0$ is then defined as

$$f(x) \approx f(x_0) + f'(x_0)(x - x_0). \quad (2.31)$$

The approximation is perfect at x_0 . The second term in (2.31) can be considered as an estimate of the change in the function value when we move from x_0 to x .

It is worthwhile to relate (2.30) to the *differential* of $f(x)$. The differential quantifies the amount of change in the function value with respect to changes in the independent variables. Mathematically, the differential of $f(x)$ is defined as

$$df(x) = f'(x)dx, \quad (2.32)$$

where dx is a small real number. The importance of using the differential lies in its ability to find the derivatives of interest. Consider the [Example A.7](#) in the textbook. Given a circle $x^2 + y^2 = r^2$ centered at the origin, we are interested in finding dy/dx subject to the constraint that both x and y stay on the circle (i.e., there is one equality constraint). Applying the differential yields

$$2xdx + 2ydy = 2rdr. \quad (2.33)$$

Setting $dr = 0$ to impose the equality constraint yields $dy/dx = -x/y$. This is the desired derivative at any point on the circle.

The linearization error of (2.31) is defined as

$$e(x) = f(x) - \left(f(x_0) + f'(x_0)(x - x_0) \right). \quad (2.34)$$

It can be shown that $e(x)$ is equal to

$$e(x) = \frac{1}{2}f''(c)(x - x_0)^2, \quad (2.35)$$

where c lies between x and x_0 , and $f''(c)$ is the second-order derivative of $f(x)$ evaluated at c . We have that

- If $f''(c) > 0$, the linearization-based approximation *underestimates* $f(x)$,
- If $f''(c) < 0$, the linearization-based approximation *overestimates* $f(x)$.

Suppose the absolute value of $f''(c)$ is upper-bounded (i.e., $|f''(c)| \leq M$), we obtain

$$|e(x)| \leq \frac{M}{2}(x - x_0)^2, \quad (2.36)$$

or equivalently,

$$-\frac{M}{2}(x - x_0)^2 \leq e(x) \leq \frac{M}{2}(x - x_0)^2. \quad (2.37)$$

The second-order approximation of $f(x)$ around x_0 is

$$f(x) \approx f(x_0) + f'(x_0)(x - x_0) + \frac{1}{2}f''(x_0)(x - x_0)^2. \quad (2.38)$$

The third term in the above approximation can be considered as an estimate of the linearization error $e(x)$. It is thus expected that the second-order approximation in (2.38) is more accurate than the linearization in (2.31). Two other observations from (2.38) are: (1) The approximation is perfect at x_0 . (2) The first-order and second-order derivatives of the approximation in (2.38) evaluated at x_0 are equal to the first-order and second-order derivatives of the true function $f(x)$ at x_0 .

Taylor's approximation theorem extends (2.31) and (2.38) to the case of using polynomials of degree N or less to approximate $f(x)$ as

$$\begin{aligned} f(x) &\approx f(x_0) + f'(x_0)(x - x_0) + \frac{1}{2!}f''(x_0)(x - x_0)^2 + \cdots + \frac{1}{N!}f^{(N)}(x_0)(x - x_0)^N \\ &= \sum_{n=0}^N \frac{1}{n!}f^{(n)}(x_0)(x - x_0)^n. \end{aligned}$$

Here, $f^{(n)}(x_0)$ denotes the n th-order derivative of $f(x)$ evaluated at x_0 . In particular, $f^{(0)}(x_0) = f(x_0)$. Again, we can show (Exercise 3.3) that when $x = x_0$, the approximation and the true function $f(x)$ have the same derivatives up to and including the order of N .

Taylor's theorem gives the approximation error in (2.39), which is

$$\begin{aligned} R_N(x) &= f(x) - \sum_{n=0}^N \frac{1}{n!}f^{(n)}(x_0)(x - x_0)^n \\ &= \frac{f^{(N+1)}(c)}{(N+1)!}(x - x_0)^{N+1}, \end{aligned} \quad (2.39)$$

where c is between x and x_0 .

We are going to show that the Taylor-series approximation given in (2.39) is *optimal* in the sense that it gives the best approximation near x_0 . Mathematically, we are going to prove that

$$|R_N(x)| < |f(x) - Q(x)|, \quad (2.40)$$

where $Q(x)$ is an arbitrary polynomials of degree N or less. Taking the limit $x \rightarrow x_0$ on the left hand side and using (2.39) yield

$$|R_N(x)| \sim \left| \frac{f^{(N+1)}(x_0)}{(N+1)!} \right| |(x - x_0)|^{N+1} \propto |(x - x_0)|^{N+1}. \quad (2.41)$$

On the other hand, we have that from Equation (2.40),

$$|f(x) - Q(x)| = \left| \sum_{n=0}^N \frac{1}{n!}f^{(n)}(x_0)(x - x_0)^n + R_N(x) - Q(x) \right|. \quad (2.42)$$

It can be seen that $\sum_{n=0}^N \frac{1}{n!} f^{(n)}(x_0)(x - x_0)^n - Q(x)$ is a polynomial of degree N or less but $R_N(x)$ is a polynomial of degree $N + 1$. Thus, Taking the limit $x \rightarrow x_0$ on both sides yields

$$|f(x) - Q(x)| \sim |a_m(x - x_0)^m| \propto |(x - x_0)|^m, \quad (2.43)$$

where $m \leq N$. In summary, we obtain

$$\lim_{x \rightarrow x_0} \frac{|R_N(x)|}{|f(x) - Q(x)|} = \lim_{x \rightarrow x_0} |(x - x_0)|^{N+1-m} = 0. \quad (2.44)$$

This completes the proof that around x_0 , the Taylor series approximation of $f(x)$ is the best among all polynomials of degree N or less.

2.5.2 Multivariate case

We now consider approximating a scalar multivariate function $f(\mathbf{x})$. Specifically, the first-order Taylor-series approximation around a particular point $(\mathbf{a}, f(\mathbf{a}))$ is given by

$$f(\mathbf{x}) \approx f(\mathbf{a}) + \nabla f(\mathbf{a})^T (\mathbf{x} - \mathbf{a}), \quad (2.45)$$

where $\nabla f(\mathbf{a})$ is the gradient of $f(\mathbf{x})$ at $\mathbf{a}$. Putting the definition of $\nabla f(\mathbf{a})$ in (2.24) transforms (2.45) into

$$f(\mathbf{x}) \approx f(\mathbf{a}) + \sum_{i=1}^n \frac{\partial f(\mathbf{x})}{\partial a_i} (x_i - a_i), \quad (2.46)$$

where $\mathbf{x} = [x_1, x_2, \dots, x_n]^T$ and $\mathbf{a} = [a_1, a_2, \dots, a_n]^T$ have been substituted. Each summand in the second term on the right hand side of (2.46) is an estimate of the change in the function value when moving along the i -th coordinate axis from a_i to x_i .

The first-order Taylor-series approximation along a certain direction $\mathbf{p}$ ($\|\mathbf{p}\|_2 = 1$) can thus be written as, from (2.45),

$$f(\mathbf{a} + \mu \mathbf{p}) \approx f(\mathbf{a}) + \mu \nabla f(\mathbf{a})^T \mathbf{p}, \quad (2.47)$$

where μ determines how far we go in the direction $\mathbf{p}$. The second term on the right hand side of (2.47) approximates the change in the function value using the *projection* along the gradient direction. Interestingly, for sufficiently small μ , if $\mathbf{p}$ is orthogonal to the gradient (i.e., $\nabla f(\mathbf{a})^T \mathbf{p} = 0$), moving along the direction $\mathbf{p}$ would not cause any change the function value.

The second-order Taylor-series approximation of $f(\mathbf{x})$ around $\mathbf{a}$ is given in the following matrix form

$$f(\mathbf{x}) \approx f(\mathbf{a}) + \nabla f(\mathbf{a})^T (\mathbf{x} - \mathbf{a}) + \frac{1}{2} (\mathbf{x} - \mathbf{a})^T \mathbf{H}_f(\mathbf{a}) (\mathbf{x} - \mathbf{a}), \quad (2.48)$$

where $\mathbf{H}_f(\mathbf{a})$ is the Hessian matrix of $f(\mathbf{a})$ evaluated at $\mathbf{a}$. Putting the definition of $\mathbf{H}_f(\mathbf{a})$ in (2.25) and using (2.18) yield

$$f(\mathbf{x}) \approx f(\mathbf{a}) + \sum_{i=1}^n \frac{\partial f(\mathbf{x})}{\partial a_i} (x_i - a_i) + \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \frac{\partial^2 f(\mathbf{x})}{\partial x_i \partial x_j} (x_i - a_i)(x_j - a_j). \quad (2.49)$$

The second-order Taylor-series approximation along a certain direction $\mathbf{p}$ ($\|\mathbf{p}\|_2 = 1$) can be written as, from (2.48),

$$f(\mathbf{a} + \mu\mathbf{p}) \approx f(\mathbf{a}) + \mu\nabla f(\mathbf{a})^T \mathbf{p} + \frac{1}{2}\mu^2 \mathbf{p}^T \mathbf{H}_f(\mathbf{a}) \mathbf{p}. \quad (2.50)$$

See [Example A.2](#) on page 541 of the textbook for an example of approximating $f(\mathbf{x})$ using (2.48).

2.6 Rate of Convergence

In this course, we mainly focus on iterative methods that are able to generate a *sequence* of solutions to a particular optimization problem. In general, we need to examine (1) whether the solution sequence converges or not, and (2) if it converges, how fast it converges. The latter characteristic is quantified using the *rate of convergence*.

Assume that the iterative technique in consideration produces a sequence of solutions denoted by $\mathbf{x}_0, \mathbf{x}_1, \mathbf{x}_2, \dots$, where the number in the subscript represents the number of iterations performed. $\mathbf{x}_0$ is normally called the *initial solution guess*. Let $\mathbf{x}^*$ be the solution that the sequence converges to. Then, $\|\mathbf{x}_k - \mathbf{x}^*\|$, which is defined using a certain vector norm, is the deviation of the solution obtained in the k -th iteration from the final solution $\mathbf{x}^*$.

The rate of convergence relates $\|\mathbf{x}_{k+1} - \mathbf{x}^*\|$ to $\|\mathbf{x}_k - \mathbf{x}^*\|$ through

$$\|\mathbf{x}_{k+1} - \mathbf{x}^*\| = \gamma_k \|\mathbf{x}_k - \mathbf{x}^*\|^p, \quad (2.51)$$

where γ_k may vary from iteration to iteration, and $p > 0$.

If $p = 1$, we have *linear* order of convergence, while $p = 2$, we have *quadratic* order of convergence. Quadratic order of convergence is a highly desired characteristic for any iterative method.

For linear convergence, the sequence of γ_k determines the speed of reduction in the norm of the deviation. For example, if $\gamma_k = 0.1$, after six iterations, the amount of deviation is reduced by a factor of 10^6 . On the other hand, if $\gamma_k = 0.9$, the deviation is reduced approximately by half ($0.9^6 \approx 0.53$). Finally, if $\gamma_k > 1$, the iterative algorithm *diverges*, as the generated solutions tend to deviate more from the final solution.

For $p \geq 1$ and $\lim_{k \rightarrow +\infty} \gamma_k = 0$, we have *superlinear* convergence, which includes quadratic convergence. But in many cases, we just have $p = 1$ and $\lim_{k \rightarrow +\infty} \gamma_k = 0$. As an example, suppose $p = 1$ and $\gamma_k = \frac{1}{(k+1)}$. After six iterations, the amount of deviation is reduced by a factor of $7! = 5040$, which is slower than the case with $p = 1$ and $\gamma_k = 0.1$ ¹. but much faster than the case with $p = 1$ and $\gamma_k = 0.9$.

To visualize the convergence behavior of an iterative algorithm, one way is to plot $\|\mathbf{x}_k - \mathbf{x}^*\|$ as a function of k . It is recommended that a logarithm scale is adopted for the y-axis. The purpose is to better reveal the small changes hard to observe with linear scales. For an example, see [Fig. 3.15](#) in the textbook.

¹With superlinear convergence under $p = 1$ and $\gamma_k = 1/(k+1)$, when k , the number of iterations performed is large enough, the rate of convergence will be faster than the case with $p = 1$ and γ_k being a positive constant γ small than 1. This can be established by showing that $\lim_{k \rightarrow +\infty} \frac{1/(k+1)!}{\gamma^k} = 0$. The proof is left as an exercise ([Exercise 3.9](#)).

In practice, the final solution $\mathbf{x}^*$ is not known. To bypass this difficulty, we evaluate the rate of convergence using

$$\frac{\|\mathbf{x}_{k+1} - \mathbf{x}_k\|}{\|\mathbf{x}_k - \mathbf{x}_{k-1}\|}. \quad (2.52)$$

The above criterion utilizes the fact that if the sequence of solutions converges, the solutions should become closer to one another when the number of iterations performed is large enough.

Following a similar idea, another way to determine approximate convergence is to use

$$\frac{\|\mathbf{x}_{k+1} - \mathbf{x}^k\|}{1 + \|\mathbf{x}_k\|}. \quad (2.53)$$

In words, when the sequence of solutions converges, the change between the solutions obtained in two consecutive iterations will be smaller, compared to the magnitude of the solution itself.

The two quantities in (2.52) and (2.53) can both be used in the implementation of practical optimization algorithms for terminating the iteration. Normally, we first select a tolerance threshold ϵ . The iteration is stopped when

$$\|\mathbf{x}_{k+1} - \mathbf{x}_k\| / \|\mathbf{x}_k - \mathbf{x}_{k-1}\| \leq \epsilon, \quad (2.54)$$

or

$$\|\mathbf{x}_{k+1} - \mathbf{x}^k\| / (1 + \|\mathbf{x}_k\|) \leq \epsilon, \quad (2.55)$$

is satisfied.

Chapter 3

Unconstrained Optimization

3.1 Introduction

This chapter focuses on the following *unconstrained* optimization problem:

$$\min_{\mathbf{x}} f(\mathbf{x}), \quad (3.1)$$

where $\mathbf{x} = [x_1, x_2, \dots, x_n]^T$ collects the continuous design variables and $f(\mathbf{x})$ is the objective function.

We are interested in developing numerical techniques that can generate a sequence of solutions $\mathbf{x}_1, \mathbf{x}_2, \dots$ to (3.1). In particular, these techniques utilize the gradient information of the objective function $f(\mathbf{x})$ to find a local minimum, starting from an initial solution guess $\mathbf{x}_0$.

In some cases, there exist bound constraints on the design variables. The constraints are often very loose. Some techniques for handling/eliminating these constraints are given in [Chapter 3.3.1](#).

3.2 Optimality Conditions

3.2.1 Necessary conditions

We define the conditions that qualify a given point $\mathbf{x}^*$ as a local minimum. We are interested in local optimality because global optimality can only be established in limited cases.

The point $\mathbf{x}^*$ is a local minimum if $f(\mathbf{x}^*) \leq f(\mathbf{x})$ for any $\mathbf{x}$ in the neighborhood of $\mathbf{x}^*$. The second-order Taylor-series approximation of the objective function $f(\mathbf{x})$ around $\mathbf{x}^*$ is, according to (2.50),

$$f(\mathbf{x}^* + \mu \mathbf{p}) \approx f(\mathbf{x}^*) + \mu \nabla f(\mathbf{x}^*)^T \mathbf{p} + \frac{1}{2} \mu^2 \mathbf{p}^T \mathbf{H}_f(\mathbf{x}^*) \mathbf{p}, \quad (3.2)$$

where $\mathbf{p}$ is an arbitrary direction vector with $\|\mathbf{p}\|_2 = 1$. For sufficiently small step size $\mu > 0$, we have that when $\mathbf{x}^*$ is a local minimum,

$$f(\mathbf{x}^*) \leq f(\mathbf{x}^* + \mu \mathbf{p}) \approx f(\mathbf{x}^*) + \mu \nabla f(\mathbf{x}^*)^T \mathbf{p}. \quad (3.3)$$

In other words, we require $\nabla f(\mathbf{x}^*)^T \mathbf{p} \geq 0$. Since $\mathbf{p}$ is an arbitrary direction vector, the gradient of the objective function at a local minimum $f(\mathbf{x}^*)$ must satisfy

$$\nabla f(\mathbf{x}^*) = \mathbf{0}. \quad (3.4)$$

This is the *first-order necessary optimality condition* for a local minimum of an unconstrained problem.

Putting (3.4) into (3.2) yields

$$f(\mathbf{x}^* + \mu \mathbf{p}) \approx f(\mathbf{x}^*) + \frac{1}{2} \mu^2 \mathbf{p}^T \mathbf{H}_f(\mathbf{x}^*) \mathbf{p}. \quad (3.5)$$

Thus, for the point $\mathbf{x}^*$ that makes the gradient of the objective function zero to be a local minimum, we require

$$\mathbf{p}^T \mathbf{H}_f(\mathbf{x}^*) \mathbf{p} \geq 0. \quad (3.6)$$

With $\mathbf{p}$ being an arbitrary direction vector, the Hessian matrix $\mathbf{H}_f(\mathbf{x}^*)$ must be at least positive semidefinite (see the definitions of Hessian in (2.25) and positive semidefinite matrices in (2.23)). This is the *second-order necessary optimality condition*.

In words, the first-order and second-order necessary optimality conditions state that: if a point $\mathbf{x}^*$ is a local minimum, we cannot find any direction that can immediately decrease the objective function (first-order necessary optimality condition) by taking a sufficiently small step along that direction; moreover, if we move slightly farther away from $\mathbf{x}^*$, the objective function will increase or at least remain the same (second-order necessary optimality condition).

3.2.2 Sufficient conditions

The sufficient conditions for a point $\mathbf{x}^*$ to be a local minimum is that

$$\nabla f(\mathbf{x}^*) = \mathbf{0}, \quad (3.7a)$$

$$\mathbf{H}_f(\mathbf{x}^*) \text{ is positive definite.} \quad (3.7b)$$

The positive definiteness of a matrix is defined in (2.22).

Positive definiteness of the Hessian guarantees that $\mathbf{x}^*$ is not a weak minimum. For a weak minimum, we can always find some directions such that $\mathbf{p}^T \mathbf{H}_f(\mathbf{x}^*) \mathbf{p}$ is zero.

For an illustration of strong and weak minima, as well as saddle point and maximum, see Fig. 4.11 in the textbook. It is important to examine Example 4.17 in the textbook as well for solving unconstrained optimization problems using (3.7).

3.3 Gradient-based Methods

The methods that will be presented focus on producing, in the current iteration (iteration k), the solution $\mathbf{x}_k$ on the basis of $\mathbf{x}_{k-1}$ obtained in the previous iteration (iteration $k-1$). This is usually achieved in two steps, where in the first step, the search direction is determined, and in the second step, searching along the search direction, also known as line search, is performed to find $\mathbf{x}_k$. Mathematically, we have

$$\mathbf{x}_k = \mathbf{x}_{k-1} + \alpha_k \mathbf{p}_k, \quad (3.8)$$

where $\mathbf{p}_k$ is the search direction and α_k is the step size found by the line search.

We are going to consider the use of gradient information, which leads to the development of gradient-based methods here¹. A graphic illustration of this type of optimization algorithms is given in Fig. 4.1 in the textbook.

¹In fact, the optimization algorithms are normally named after the method used to find the search direction.

Recall that as pointed out in [Chapter 2.3.1](#), each component of the gradient $\nabla f(\mathbf{x})$ gives the rate of change of the objective function $f(\mathbf{x})$ at $\mathbf{x}$ (see an illustration in [Fig. 4.2](#) in the textbook and note that the arrowed line pointing to the direction of increasing the objective function).

In fact, the gradient points in the direction of the *fastest* function increase from the current point. To verify this, consider the first-order Taylor-series approximation of the objective function (also called the *directional gradient*, see [Figs. 4.5 and 4.6](#) in the textbook for an illustration), which is, from [\(2.47\)](#),

$$f(\mathbf{x} + \mu \mathbf{p}) \approx f(\mathbf{x}) + \mu \nabla f(\mathbf{x})^T \mathbf{p}. \quad (3.9)$$

We have that $\nabla f(\mathbf{x})^T \mathbf{p} = \|\nabla f(\mathbf{x})\|_2 \|\mathbf{p}\|_2 \cos(\theta)$, where $\mathbf{p}$ is an arbitrary direction vector and θ is the angle between $\nabla f(\mathbf{x})$ and $\mathbf{p}$. Therefore, the direction vector that maximizes the increase in the objective function is the one in the *same* direction of the gradient (i.e., $\cos(\theta) = 1$). In other words, the gradient gives the direction of the fastest function increase.

It can also be noticed from [\(3.9\)](#) that if $\theta \in (-\pi/2, \pi/2)$, moving along the directional gradient still leads to a larger objective function value. For $|\theta| = \pi/2$, moving along the directional gradient gives the *same* function value under sufficiently small step size μ . When $\theta = \pi$, this directional gradient gives the direction of the steepest descent.

3.3.1 Steepest descent

A simple and intuitive way to determine the search direction is to directly use the inverse of the gradient at $\mathbf{x}_{k-1}$, because $-\nabla f(\mathbf{x}_{k-1})$ gives the direction of the steepest decrease. This method is called *steepest descent* or *gradient descent*. In this method, we can set

$$\mathbf{p}_k = -\nabla f(\mathbf{x}_{k-1}). \quad (3.10)$$

or more commonly,

$$\mathbf{p}_k = -\frac{\nabla f(\mathbf{x}_{k-1})}{\|\nabla f(\mathbf{x}_{k-1})\|_2}. \quad (3.11)$$

The normalization in [\(3.11\)](#) can mitigate the effect of the varying design variable magnitude on the magnitude of the gradient elements. Another effective way to mitigate this is to scale the design variables (see [Tip 4.4](#) and [Example 4.19](#) in the textbook). The rule of thumb is to try making the objective function and all design variables have an order of magnitude of 1.

The steepest gradient may be highly inefficient, especially when the curvature of the objective function varies greatly with the direction, which makes the gradient a poor representation of the function behavior beyond a small neighborhood. In this case, the steepest gradient could show a zigzagging pattern (see [Example 4.10](#) in the textbook for an illustration).

The above phenomenon can be explained mathematically. Suppose an exact line search strategy is adopted, which means that the value of α_k is selected to minimize the objective function along the direction $\mathbf{p}_k$. In other words, we find α_k through solving

$$\frac{\partial f(\mathbf{x}_{k-1} + \alpha_k \mathbf{p}_k)}{\partial \alpha_k} = 0. \quad (3.12)$$

By Chain rule of the derivative, we have that

$$\begin{aligned}\frac{\partial f(\mathbf{x}_{k-1} + \alpha_k \mathbf{p}_k)}{\partial \alpha_k} &= \left(\frac{\partial f(\mathbf{x}_{k-1} + \alpha_k \mathbf{p}_k)}{\partial \mathbf{x}_{k-1} + \alpha_k \mathbf{p}_k} \right)^T \frac{\partial \mathbf{x}_{k-1} + \alpha_k \mathbf{p}_k}{\partial \alpha_k} \\ &= \nabla f(\mathbf{x}_{k-1} + \alpha_k \mathbf{p}_k)^T \mathbf{p}_k \\ &= \nabla f(\mathbf{x}_{k+1})^T \mathbf{p}_k.\end{aligned}\tag{3.13}$$

In other words, each search direction is orthogonal to the previous one (if (3.13) can be solved *exactly*). Again, to mitigate the zigzagging behavior, scaling the design variables sometimes is an effective technique.

3.3.2 Conjugate gradient

It is known that the gradient descent can perform poorly in narrow valleys. The *conjugate gradient* method overcomes this disadvantage by the following insights obtained from solving an unconstrained quadratic optimization problem

$$f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T \mathbf{A} \mathbf{x} - \mathbf{b}^T \mathbf{x}.\tag{3.14}$$

where $\mathbf{A}$ is a positive definite matrix. The above problem has a unique global minimum that can be found by solving

$$\mathbf{A} \mathbf{x} = \mathbf{b}.\tag{3.15}$$

Taking inverse of $\mathbf{A}$ can be computationally cumbersome, because this has a complexity of $\mathcal{O}(n^3)$, where n is the dimensionality of $\mathbf{x}$. Instead, we are going to determine, using the conjugate gradient method, a sequence of solutions $\mathbf{x}_0, \mathbf{x}_1, \dots$ that converges to the solution satisfying (3.15)².

We can notice that if $\mathbf{A}$ is diagonal, the contours of the objective function $f(\mathbf{x})$ in (3.14) would be elliptical and their axes are aligned with the coordinate axes. As a result, we can start at any point and are able to converge to the desired global minimum through successively performing an exact line search as in (3.13) in each coordinate direction for a total of n line searches. See Fig. 4.38 in the textbook for an illustration.

For the more general case where $\mathbf{A}$ is not diagonal, the contours of the objective function $f(\mathbf{x})$ in (3.14) would still be elliptical but their axes are no longer aligned with the coordinate axes. In this case, we can still converge to the desired global minimum through successively performing an exact line search as in (3.13) in each principle curvature direction (i.e., the eigenvector of $\mathbf{A}$, see Chapter 2.3.2) for a total of n line searches. See Fig. 4.40 in the textbook for an illustration. However, computing the eigenvectors of $\mathbf{A}$ is still quite costly. The complexity is again on the order of $\mathcal{O}(n^3)$.

A significant property of the eigenvectors of a symmetric $\mathbf{A}$ is that they are *conjugate* with respect to $\mathbf{A}$. Mathematically, we have that they satisfy

$$\mathbf{v}_i^T \mathbf{A} \mathbf{v}_j = 0,\tag{3.16}$$

where $\mathbf{v}_i$ and $\mathbf{v}_j$ are different eigenvectors of $\mathbf{A}$. The conjugate gradient method is a more convenient way to find a sequence of conjugate search directions for the problem in (3.14).

²The conjugate gradient method presented can also be applied to solve the linear equations $\mathbf{A} \mathbf{x} = \mathbf{b}$, where $\mathbf{A}$ is positive definite and has large dimensionality.

The conjugate gradient method starts with an initial solution guess $\mathbf{x}_0$. Performing an exact line search along the direction of the steepest descent $\mathbf{p}_0 = -\nabla f(\mathbf{x}_0) = -(\mathbf{A}\mathbf{x}_0 - \mathbf{b})$ yields the step size ([Exercise 4.1](#))

$$\alpha_0 = -\frac{\mathbf{p}_0^T(\mathbf{A}\mathbf{x}_0 - \mathbf{b})}{\mathbf{p}_0^T\mathbf{A}\mathbf{p}_0} = -\frac{\mathbf{p}_0^T\nabla f(\mathbf{x}_0)}{\mathbf{p}_0^T\mathbf{A}\mathbf{p}_0}. \quad (3.17)$$

The next solution is thus given by $\mathbf{x}_1 = \mathbf{x}_0 + \alpha_0\mathbf{p}_0$. The next search direction $\mathbf{p}_1$ is found by setting it to be

$$\mathbf{p}_1 = -\nabla f(\mathbf{x}_1) + \beta_1\mathbf{p}_0, \quad (3.18)$$

and requiring that $\mathbf{p}_0$ and $\mathbf{p}_1$ are conjugate with respect to $\mathbf{A}$. In other words, the new search direction $\mathbf{p}_1$ is a linear combination of the previous search direction $\mathbf{p}_0$ and the direction of the steepest descent at the current solution $\mathbf{x}_1$. β_1 is found through applying the definition of conjugacy in ([3.16](#)), which is

$$\beta_1 = \frac{\mathbf{p}_0^T\mathbf{A}\nabla f(\mathbf{x}_1)}{\mathbf{p}_0^T\mathbf{A}\mathbf{p}_0}. \quad (3.19)$$

We are going to repeat the above process until the residual $r_k = \mathbf{A}\mathbf{x}_k - \mathbf{b}$ is sufficiently small (i.e., $\|r_k\|_\infty < \tau$). The corresponding conjugate gradient algorithm is summarized in [Algorithm B.2 in Appendix B.4.2](#) of the textbook. The major steps in the k -th iteration ($k \geq 1$) are summarized here as follows:

$$\alpha_k = -\frac{\mathbf{p}_{k-1}^T\nabla f(\mathbf{x}_{k-1})}{\mathbf{p}_{k-1}^T\mathbf{A}\mathbf{p}_{k-1}}, \quad (3.20a)$$

$$\mathbf{x}_k = \mathbf{x}_{k-1} + \alpha_k\mathbf{p}_{k-1}, \quad (3.20b)$$

$$\beta_k = \frac{\mathbf{p}_{k-1}^T\mathbf{A}\nabla f(\mathbf{x}_k)}{\mathbf{p}_{k-1}^T\mathbf{A}\mathbf{p}_{k-1}}, \quad (3.20c)$$

$$\mathbf{p}_k = -\nabla f(\mathbf{x}_k) + \beta_k\mathbf{p}_{k-1}. \quad (3.20d)$$

Two options to compute β_k in ([3.20c](#)) exist. The first one is the *Fletcher-Reeves* formula, where β_k is calculated using

$$\beta_k = \frac{\nabla f(\mathbf{x}_k)^T\nabla f(\mathbf{x}_k)}{\nabla f(\mathbf{x}_{k-1})^T\nabla f(\mathbf{x}_{k-1})}. \quad (3.21)$$

The second approach is the *Polak-Ribiere* formula, which is given by

$$\beta_k = \frac{\nabla f(\mathbf{x}_k)^T(\nabla f(\mathbf{x}_k) - \nabla f(\mathbf{x}_{k-1}))}{\nabla f(\mathbf{x}_{k-1})^T\nabla f(\mathbf{x}_{k-1})}. \quad (3.22)$$

The Fletcher-Reeves formula is theoretically established in [Appendix B.4.2](#) in the textbook. Some insights can be obtained by examining ([3.21](#)). It can be seen that with the Fletcher-Reeves formula, β_k is always positive. Therefore, the search direction $\mathbf{p}_k$ is guaranteed to be a direction, along which the objective function can be decreased ([Exercise 4.2](#)). On the other hand, with the Polak-Ribiere formula, β_k may become negative, when the cost function is nonlinear with respect to the design variables. To guarantee that $\mathbf{p}_k$ is still a descent direction, we can rectify β_k using

$$\beta_k = \max(0, \beta_k). \quad (3.23)$$

The method of conjugate gradient can be applied to minimize nonlinear objective functions. The only requirement is that the objective function is smooth and its gradient is calculable. The major steps in the k -th iteration can be summarized as follows.

- Perform a line search (see [Chapter 4.3.4](#)) to find the step size α_k along the search direction $\mathbf{p}_{k-1}$,
- Find the current candidate solution and update the search direction as in [\(3.20b\)](#) and [\(3.20d\)](#), where β_k is calculated using either [\(3.21\)](#) or [\(3.22\)](#),
- If the Fletcher-Reeves formula [\(3.21\)](#) is used, reset the search direction $\mathbf{p}_k$ to the steepest descent direction every n iterations. If the Polak-Ribiere formula is used, the rectification operation in [\(3.23\)](#) can be adopted to reset the search direction.

It is found empirically that adopting the Polak-Ribiere formula leads to faster convergence than with the Fletcher-Reeves formula.

3.3.3 Newton's method

The gradient descent and conjugate gradient methods both utilize the gradient information (first-order information) only. Line search for step size determination is thus needed to ensure the decrease of the objective function through moving along these search directions. On the other hand, Newton's method explores the first-order and second-order information of the objective function around a point. It can provide an estimate of the step size, besides the search direction.

Newton's method approximates the objective function $f(\mathbf{x})$ around the solution obtained in the previous iteration (iteration $k-1$), $\mathbf{x}_{k-1}$, using the second-order Taylor-series expansion given in [\(2.48\)](#). Mathematically, we have

$$f(\mathbf{x}) \approx f(\mathbf{x}_{k-1}) + \nabla f(\mathbf{x}_{k-1})^T (\mathbf{x} - \mathbf{x}_{k-1}) + \frac{1}{2} (\mathbf{x} - \mathbf{x}_{k-1})^T \mathbf{H}_f(\mathbf{x}_{k-1}) (\mathbf{x} - \mathbf{x}_{k-1}). \quad (3.24)$$

Taking the derivative of the right-hand side of [\(3.24\)](#) with respect to $\mathbf{x}$ and setting the result to zero yield

$$\nabla f(\mathbf{x}_{k-1}) + \mathbf{H}_f(\mathbf{x}_{k-1})(\mathbf{x} - \mathbf{x}_{k-1}) = 0. \quad (3.25)$$

If the Hessian $\mathbf{H}_f(\mathbf{x}_{k-1})$ (see the definition of Hessian in [\(2.25\)](#)) is invertible, the solution in the k -th iteration can be found through evaluating

$$\mathbf{x} = \mathbf{x}_{k-1} - \mathbf{H}_f(\mathbf{x}_{k-1})^{-1} \nabla f(\mathbf{x}_{k-1}). \quad (3.26)$$

It can be seen that the search direction and step size is obtained through rotating and stretching the gradient at $\mathbf{x}_{k-1}$ using the Hessian.

One important characteristic of Newton's method is that if the original objective function is quadratic (i.e., $f(\mathbf{x})$ has the form $f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T \mathbf{A} \mathbf{x} + \mathbf{b}^T \mathbf{x} + c$, where $\mathbf{A}$ is positive definite), Newton's method, starting at *any* point $\mathbf{x}_0$, converges to the minimum in a one step ([Exercise 4.3](#)). An example is shown in [Fig. 4.45](#) in the textbook, which illustrates that Newton's method is *scale invariant*.

In practice, Newton's method could suffer from a few issues. One is that Newton step in [\(3.26\)](#) may necessarily lead to decreasing the objective function. In particular, if the Hessian in [\(3.24\)](#), $\mathbf{H}_f(\mathbf{x}_{k-1})$, is not positive definite, Newton's method in this case may even increase the objective function. One typical approach is again to apply the line search, which results in the following solution update

$$\mathbf{x} = \mathbf{x}_{k-1} - \alpha_k \mathbf{H}_f(\mathbf{x}_{k-1})^{-1} \nabla f(\mathbf{x}_{k-1}). \quad (3.27)$$

The step size α_k can be initialized to 1, when starting the line search.

Levenberg-Marquardt Algorithm

Consider a quadratic objective function

$$f(\mathbf{x}) = \frac{1}{2}(\mathbf{b} - \mathbf{g}(\mathbf{x}))^T(\mathbf{b} - \mathbf{g}(\mathbf{x})). \quad (3.28)$$

Minimizing it is in fact a nonlinear least squares (NLS) problem. Gradient descent and conjugate gradient methods can be applied to this type of problem. But for this particular type of problems, techniques that explore the specific structure of $f(\mathbf{x})$ in (6.17) do exist. They are *Gauss-Newton* algorithm and *Levenberg-Marquardt* algorithm.

The idea of the Gauss-Newton algorithm is to approximate the nonlinear function $\mathbf{g}(\mathbf{x})$ using the first-order Taylor-series expansion (see (2.45)) as

$$\mathbf{g}(\mathbf{x}) \approx \mathbf{g}(\mathbf{x}_{k-1}) + \mathbf{J}_g(\mathbf{x}_{k-1})(\mathbf{x} - \mathbf{x}_{k-1}). \quad (3.29)$$

Here, $\mathbf{J}_g(\mathbf{x}_{k-1})$ denotes the *Jacobian* of the multi-variate function $\mathbf{g}(\mathbf{x})$ around the solution obtained in the previous iteration $\mathbf{x}_{k-1}$. It is defined as

$$\mathbf{J}_g(\mathbf{x}_{k-1}) = \begin{bmatrix} \nabla g_1(\mathbf{x}_{k-1})^T \\ \nabla g_2(\mathbf{x}_{k-1})^T \\ \vdots \\ \nabla g_m(\mathbf{x}_{k-1})^T \end{bmatrix}, \quad (3.30)$$

where $g_j(\mathbf{x})$ is the j -th component of $\mathbf{g}(\mathbf{x})$.

Putting (3.29) into (6.17) yields

$$f(\mathbf{x}) \approx \frac{1}{2}(\mathbf{b} - \mathbf{g}(\mathbf{x}_{k-1}) - \mathbf{J}_g(\mathbf{x}_{k-1})(\mathbf{x} - \mathbf{x}_{k-1}))^T(\mathbf{b} - \mathbf{g}(\mathbf{x}_{k-1}) - \mathbf{J}_g(\mathbf{x}_{k-1})(\mathbf{x} - \mathbf{x}_{k-1})). \quad (3.31)$$

The objective function now becomes a quadratic function with respect to the design variables $\mathbf{x}$. Following the process that leads to (3.25) and (3.26) yields the solution at the current iteration, which is (Exercise 4.4)

$$\mathbf{x}_k = \mathbf{x}_{k-1} + (\mathbf{J}_g(\mathbf{x}_{k-1})^T \mathbf{J}_g(\mathbf{x}_{k-1}))^{-1} \mathbf{J}_g(\mathbf{x}_{k-1})^T (\mathbf{b} - \mathbf{g}(\mathbf{x}_{k-1})). \quad (3.32)$$

Comparing (3.32) with (3.26) reveals that the Gauss-Newton update is similar, to some extent, to the update of Newton's method. But the Gauss-Newton method uses the first-order information only. In fact, with the Newton's method, the iteration should be given in (3.26) with the gradient and Hessian at $\mathbf{x}_{k-1}$ equal to (Exercise 4.5)

$$\nabla f(\mathbf{x}_{k-1}) = -\mathbf{J}_g(\mathbf{x}_{k-1})^T (\mathbf{b} - \mathbf{g}(\mathbf{x}_{k-1})). \quad (3.33)$$

and

$$\mathbf{H}_f(\mathbf{x}_{k-1}) = \mathbf{J}_g(\mathbf{x}_{k-1})^T \mathbf{J}_g(\mathbf{x}_{k-1}) - \sum_{j=1}^m (b_j - g_j(\mathbf{x}_{k-1})) \mathbf{H}_{g_j}(\mathbf{x}_{k-1}), \quad (3.34)$$

where $\mathbf{H}_{g_j}(\mathbf{x}_{k-1})$ is the Hessian of $g_j(\mathbf{x})$ at $\mathbf{x}_{k-1}$ and it is defined as, according to see (2.25),

$$\mathbf{H}_{g_j}(\mathbf{x}) = \frac{\partial^2 g_j(\mathbf{x})}{\partial \mathbf{x} \partial \mathbf{x}^T}. \quad (3.35)$$

It can be seen that the Gauss-Newton algorithm can be considered as a *modified* version of the Newton's method if the second term on the right hand side of (3.34) is ignored³. This approximation would be effective if the residual $\mathbf{b} - \mathbf{g}(\mathbf{x}_{k-1})$ is small. Otherwise, the Gauss-Newton update in (3.32) may be inaccurate.

To address this aspect, applying the line search is a straightforward way. Another strategy that can be adopted is the Levenberg-Marquardt algorithm. The idea comes from the following insight. If we replace the second term on the right hand side of (3.34) with $\mu \mathbf{I}$, where $\mathbf{I}$ is a $n \times n$ identity matrix. The Newton's algorithm in (3.26) becomes

$$\mathbf{x}_k = \mathbf{x}_{k-1} + (\mathbf{J}_g(\mathbf{x}_{k-1})^T \mathbf{J}_g(\mathbf{x}_{k-1}) + \mu \mathbf{I})^{-1} \mathbf{J}_g(\mathbf{x}_{k-1})^T (\mathbf{b} - \mathbf{g}(\mathbf{x}_{k-1})). \quad (3.36)$$

If $\mu = 0$, the above iteration reduces to the Gauss-Newton method. If $\mu \rightarrow +\infty$, the Newton's algorithm becomes the steepest gradient with a step size of $1/\mu$. The introduction of μ provides the chance to alternate between the Gauss-Newton's method and steepest descent algorithm.

The Levenberg-Marquardt algorithm replaces $\mu \mathbf{I}$ using $\mu \mathbf{D}$, where

$$\mathbf{D} = \text{diag}(\mathbf{J}_g(\mathbf{x}_{k-1})^T \mathbf{J}_g(\mathbf{x}_{k-1})). \quad (3.37)$$

$\text{diag}(\mathbf{A})$ is an operation that generates a diagonal matrix whose diagonal elements are from the diagonal of $\mathbf{A}$. Roughly speaking, the diagonal elements of $\mathbf{D}$ are the curvature of the objective function along each coordinate axis (see Chapter 2.3.2 for the definition of curvature).

The value of μ is determined as follows. If the current iteration leads to a decrease in the objective function, we may scale down μ using a constant factor $\rho < 1$ using $\mu \leftarrow \mu\rho$. This corresponds to the case that the residual becomes smaller, and the Gauss-Newton algorithm is effective. Otherwise, we keep increasing μ by $\mu \leftarrow \mu/\rho$ until a decrease in the objective function is obtained. This corresponds to applying the gradient descent with a very small step size to guarantee the improvement in the objective function. For an summary of the Levenburg-Marquardt algorithm, see Algorithm 10.3 in the textbook.

Quasi-Newton Methods

Newton's method in (3.26) requires the computation of the Hessian, which could be difficult in some applications. On the other hand, the Gauss-Newton algorithm or Levenburg-Marquardt algorithm only needs the gradient information, but their application is limited to the NLS problem, as their development exploits the specific structure of the NLS objective function. Quasi-Newton methods attempt to address these drawbacks through utilizing the gradient information obtained in each iteration only to build an approximation of the Hessian.

The development of quasi-Newton methods again starts with the quadratic approximation of the objective function around the solution from the previous iteration $\mathbf{x}_{k-1}$

$$f(\mathbf{x}) \approx f(\mathbf{x}_{k-1}) + \nabla f(\mathbf{x}_{k-1})^T (\mathbf{x} - \mathbf{x}_{k-1}) + \frac{1}{2} (\mathbf{x} - \mathbf{x}_{k-1})^T \tilde{\mathbf{H}}_f(\mathbf{x}_{k-1}) (\mathbf{x} - \mathbf{x}_{k-1}). \quad (3.38)$$

Compared with the quadratic approximation adopted by the Newton's method (see (3.24)), the true Hessian $\mathbf{H}_f(\mathbf{x}_{k-1})$ is now replaced by its *estimate* $\tilde{\mathbf{H}}_f(\mathbf{x}_{k-1})$.

³An advantage of the Gauss-Newton algorithm is that it does not need to calculate the Hessian explicitly.

The update of the quasi-Newton solution is similar to (3.27) with $\mathbf{H}_f(\mathbf{x}_{k-1})$ replaced by $\tilde{\mathbf{H}}_f(\mathbf{x}_{k-1})$, which is

$$\mathbf{x}_k = \mathbf{x}_{k-1} - \alpha_k \tilde{\mathbf{H}}_f(\mathbf{x}_{k-1})^{-1} \nabla f(\mathbf{x}_{k-1}). \quad (3.39)$$

The step size α_k needs to be found by applying the line search along the search direction $\mathbf{p}_k = -\tilde{\mathbf{H}}_f(\mathbf{x}_{k-1})^{-1} \nabla f(\mathbf{x}_{k-1})$ and it must satisfy the strong Wolf condition (see Chapter 4.3).

The quasi-Newton algorithm also needs to compute the estimate of the Hessian at $\mathbf{x}_k$ to enable the next iteration (i.e., iteration $k+1$). The updated Hessian has the form

$$\tilde{\mathbf{H}}_f(\mathbf{x}_k) = \tilde{\mathbf{H}}_f(\mathbf{x}_{k-1}) + \Delta \tilde{\mathbf{H}}_f(\mathbf{x}_k, \mathbf{x}_{k-1}). \quad (3.40)$$

The update $\Delta \tilde{\mathbf{H}}_f(\mathbf{x}_k, \mathbf{x}_{k-1})$ depends on the gradients of the objective function at $\mathbf{x}_k$ and $\mathbf{x}_{k-1}$. It is established based on the following idea: with the estimated Hessian $\tilde{\mathbf{H}}_f(\mathbf{x}_k)$, the quadratic approximation of the objective function at $\mathbf{x}_k$ matches the actual gradients of the objective function at $\mathbf{x}_k$ and $\mathbf{x}_{k-1}$. Mathematically, the objective function is approximated at $\mathbf{x}_k$ using

$$f(\mathbf{x}) \approx f(\mathbf{x}_k) + \nabla f(\mathbf{x}_k)^T (\mathbf{x} - \mathbf{x}_k) + \frac{1}{2} (\mathbf{x} - \mathbf{x}_k)^T \tilde{\mathbf{H}}_f(\mathbf{x}_k) (\mathbf{x} - \mathbf{x}_k). \quad (3.41)$$

Its gradient at $\mathbf{x}_k$ is $\nabla f(\mathbf{x}_k)$, which is exactly the true gradient of the objective function at $\mathbf{x}_k$. The gradient at $\mathbf{x}_{k-1}$ is equal to (Exercise 4.6) $\nabla f(\mathbf{x}_k) + \tilde{\mathbf{H}}_f(\mathbf{x}_k)(\mathbf{x}_{k-1} - \mathbf{x}_k)$. We require that

$$\nabla f(\mathbf{x}_{k-1}) = \nabla f(\mathbf{x}_k) + \tilde{\mathbf{H}}_f(\mathbf{x}_k)(\mathbf{x}_{k-1} - \mathbf{x}_k). \quad (3.42)$$

Or equivalently,

$$\tilde{\mathbf{H}}_f(\mathbf{x}_k)(\mathbf{x}_k - \mathbf{x}_{k-1}) = \nabla f(\mathbf{x}_k) - \nabla f(\mathbf{x}_{k-1}). \quad (3.43)$$

(3.43) is called the *secant equation*. It is consistent with the intuition behind the product of the Hessian with a vector: the change in the gradient due to moving from $\mathbf{x}_{k-1}$ to $\mathbf{x}_k$ is given by multiplying the Hessian with the vector $\mathbf{x}_k - \mathbf{x}_{k-1}$ (also see Chapter 2.3.2). For notation simplicity, we define $\mathbf{y}_k = \nabla f(\mathbf{x}_k) - \nabla f(\mathbf{x}_{k-1})$ and $\mathbf{s}_k = \mathbf{x}_k - \mathbf{x}_{k-1}$, which transforms the secant equation into

$$\tilde{\mathbf{H}}_f(\mathbf{x}_k) \mathbf{s}_k = \mathbf{y}_k. \quad (3.44)$$

We also need the estimated Hessian to be positive definite to ensure that the search direction computed using the quadratic approximation always points to the descent direction⁴. A necessary condition is thus, from the secant equation in (3.44),

$$0 < \mathbf{s}_k^T \tilde{\mathbf{H}}_f(\mathbf{x}_k) \mathbf{s}_k = \mathbf{s}_k^T \mathbf{y}_k. \quad (3.45)$$

This is called the *curvature condition*.

The secant equation is a linear system of n equations. But to find the update $\Delta \tilde{\mathbf{H}}_f(\mathbf{x}_k, \mathbf{x}_{k-1})$ in (3.40) with $n(n+1)/2$ unknowns, it is not sufficient. To address this difficulty, we resort to the idea of low-rank update. Specifically, it is assumed that the updated Hessian $\mathbf{H}_f(\mathbf{x}_k)$ is closest to $\mathbf{H}_f(\mathbf{x}_{k-1})$ in the sense that the update $\Delta \tilde{\mathbf{H}}_f(\mathbf{x}_k, \mathbf{x}_{k-1})$

⁴Exercise 4.7: Show that the search direction at the $(k-1)$ th iteration, which is, according to (3.38), $\mathbf{p}_k = -\tilde{\mathbf{H}}_f(\mathbf{x}_{k-1})^{-1} \nabla f(\mathbf{x}_{k-1})$, is a descent direction.

has low rank. This greatly reduces the number of unknowns to be determined. The rationale behind this idea is that at each iteration, we only obtain one more gradient of the objective function. Therefore, the amount of change in the Hessian should not be significant.

Among the proposed low-rank quasi-Newton updates, the one independently developed by Broyden, Fletcher, Goldfarb and Shanno, and commonly referred to as the BFGS update is considered the most effective one. We shall focus on the BFGS update here and other low-rank quasi-Newton updates are briefly discussed in [Appendix C](#) in the textbook.

To ensure that the estimated Hessian is positive definite, the BFGS update takes the form

$$\Delta \tilde{\mathbf{H}}_f(\mathbf{x}_k, \mathbf{x}_{k-1}) = a \cdot \mathbf{u}\mathbf{u}^T + b \cdot \mathbf{v}\mathbf{v}^T, \quad (3.46)$$

where $\mathbf{u}$ and $\mathbf{v}$ are linearly independent column vectors, and a and b are coefficients to be decided. It can be seen that the BFGS update is a positive semidefinite matrix of rank 2. Putting (3.46) into (3.40) and using the secant equation in (3.44), we have

$$\tilde{\mathbf{H}}_f(\mathbf{x}_{k-1})\mathbf{s}_k + a \cdot \mathbf{u}\mathbf{u}^T\mathbf{s}_k + b \cdot \mathbf{v}\mathbf{v}^T\mathbf{s}_k = \mathbf{y}_k. \quad (3.47)$$

In the BFGS update, $\mathbf{u} = \mathbf{y}_k$ and $\mathbf{v} = \tilde{\mathbf{H}}_f(\mathbf{x}_{k-1})\mathbf{s}_k$. Substituting these results into (3.47) and re-arranging yield

$$\mathbf{y}_k(1 - a \cdot \mathbf{y}_k^T\mathbf{s}_k) = \tilde{\mathbf{H}}_f(\mathbf{x}_{k-1})\mathbf{s}_k(1 + b \cdot \mathbf{s}_k^T\tilde{\mathbf{H}}_f(\mathbf{x}_{k-1})\mathbf{s}_k). \quad (3.48)$$

In general, $\mathbf{y}_k$ and $\tilde{\mathbf{H}}_f(\mathbf{x}_{k-1})\mathbf{s}_k$ are not parallel. Therefore, for the equality in (3.48) to hold, we require

$$a = \frac{1}{\mathbf{y}_k^T\mathbf{s}_k}, \quad b = -\frac{1}{\mathbf{s}_k^T\tilde{\mathbf{H}}_f(\mathbf{x}_{k-1})\mathbf{s}_k}. \quad (3.49)$$

The BFGS update is thus given by

$$\tilde{\mathbf{H}}_f(\mathbf{x}_k) = \tilde{\mathbf{H}}_f(\mathbf{x}_{k-1}) + \frac{1}{\mathbf{y}_k^T\mathbf{s}_k}\mathbf{y}_k\mathbf{y}_k^T - \frac{1}{\mathbf{s}_k^T\tilde{\mathbf{H}}_f(\mathbf{x}_{k-1})\mathbf{s}_k}\tilde{\mathbf{H}}_f(\mathbf{x}_{k-1})\mathbf{s}_k\mathbf{s}_k^T\tilde{\mathbf{H}}_f(\mathbf{x}_{k-1})^T. \quad (3.50)$$

Combining (3.50) and (3.39) gives the BFGS quasi-Newton method for continuous unconstrained optimization.

Note that when implementing (3.39), the inverse of the estimated Hessian needs to be found. With BFGS update, an efficient approach that updates the inverse of the estimated Hessian can be established through invoking the Woodbury matrix identity (see [Appendix C.3](#) in the textbook). In particular, let $\tilde{\mathbf{V}}_k$ be the inverse of $\tilde{\mathbf{H}}_f(\mathbf{x}_k)$. $\tilde{\mathbf{V}}_k$ is related to $\tilde{\mathbf{V}}_{k-1}$ via

$$\tilde{\mathbf{V}}_k = (1 - \sigma_k\mathbf{s}_k\mathbf{y}_k^T)\tilde{\mathbf{V}}_{k-1}(1 - \sigma_k\mathbf{s}_k\mathbf{y}_k^T) + \sigma_k\mathbf{s}_k\mathbf{s}_k^T, \quad (3.51)$$

where $\sigma_k = 1/\mathbf{y}_k^T\mathbf{s}_k$. With $\tilde{\mathbf{V}}_k$, the search direction can be easily computed using $-\tilde{\mathbf{V}}_k\nabla f(\mathbf{x}_k)$.

The BFGS algorithm starts with $\tilde{\mathbf{V}}_0 = \mathbf{I}$, which corresponds to the first iteration being gradient descent. In the practical implementation, we might need to reset $\tilde{\mathbf{V}}_k$ from time to time. This is because the curvature information accumulated far from the current solution may be irrelevant and even leads to slow convergence. If the angle between the current gradient $\nabla f(\mathbf{x}_k)$ and search direction $-\tilde{\mathbf{V}}_k\nabla f(\mathbf{x}_k)$, which can be calculated

by their inner product, is smaller than a particular threshold, reset $\tilde{\mathbf{V}}_k$ to the identity matrix. In words, if the search direction becomes close to the gradient direction, which could lead to increase in the objective function, we choose to apply the gradient descent in the current iteration and reset the BFGS update.

The BFGS algorithm is summarized in [Algorithm 4.7](#) in the textbook.

3.3.4 Line Search

Line search determines the step size α_k after the search direction $\mathbf{p}_k$ at the solution $\mathbf{x}_{k-1}$ is found. To guarantee the decrease of the objective function, $\mathbf{p}_k$ needs to a descent direction such that $\mathbf{p}_k^T \nabla f(\mathbf{x}_{k-1}) < 0$. Line search addresses the following question: how far to move along the direction $\mathbf{p}_k$. After line search, the next solution is computed using $\mathbf{x}_k = \mathbf{x}_{k-1} + \alpha_k \mathbf{p}_k$ (see [Fig. 4.16](#) in the textbook for an illustration).

Line search is usually *independent* of the method for choosing the search direction and thus, it can be combined with *any* search direction determination method.

Along the search direction $\mathbf{p}_k$, the objective function has a value given by $f(\mathbf{x}_{k-1} + \alpha_k \mathbf{p}_k)$. Because both $\mathbf{x}_{k-1}$ and $\mathbf{p}_k$ are known, the multivariate objective function now depends on the value of the step size α_k only. That is, it now becomes a univariate function, which corresponds to a one-dimensional slice of the n -dimensional input space (see [Fig. 4.17](#) in the textbook for an illustration).

The *realistic* goal of the line search is NOT to find the minimum of $f(\mathbf{x}_{k-1} + \alpha_k \mathbf{p}_k)$. Instead, it aims at locating a point that is ‘good enough’ using the least number of evaluations of the objective function. Searching for the value that minimizes $f(\mathbf{x}_{k-1} + \alpha_k \mathbf{p}_k)$ may require many iterations but only result in marginal objective function improvement.

To simplify the following presentation, we define

$$\phi(\alpha_k) = f(\mathbf{x}_{k-1} + \alpha_k \mathbf{p}_k), \quad (3.52)$$

to make it explicit that line search involves the manipulation of a univariate function. It is evident that α_k corresponds to the start of the line search, since $\phi(0) = f(\mathbf{x}_{k-1})$ is the objective function value at the previous solution. More importantly, we have that

$$\phi'(\alpha_k) = \nabla f(\mathbf{x}_{k-1} + \alpha_k \mathbf{p}_k)^T \mathbf{p}_k. \quad (3.53)$$

$\phi'(\alpha_k)$ is called the directional derivative along the search direction $\mathbf{p}_k$ (see also [\(2.47\)](#)). If $\phi'(\alpha_k)$ is positive, this indicates that increasing the step size over α_k would increase the objective function. On the other hand, if $\phi'(\alpha_k)$ is negative, increasing the step size over α_k would decrease the objective function. At $\alpha_k = 0$, we have $\phi(0) < 0$, since $\mathbf{p}_k$ is a descent direction, which guarantees that moving from $\mathbf{x}_{k-1}$ can always improve the objective function (see [Fig. 4.20](#) for an illustration).

Sufficient decrease condition

To find a ‘good enough’ solution, we require that the corresponding step size satisfies the sufficient decrease condition, also known as *Armijo condition*,

$$\phi(\alpha_k) \leq \phi(0) + \mu_1 \alpha_k \phi'(0), \quad (3.54)$$

where $0 < \mu_1 < 1$ is a positive constant. We usually set μ_1 to be 10^{-4} . In [\(3.54\)](#), $\alpha_k \phi'(0)$ is the *expected* decrease in the objective function, assuming that the function continues

at the same slope of $\phi'(0)$. The factor μ_1 is less than one, indicating that we consider a solution to be a good one, as long as it achieves a very small *fraction* of the expected decrease (see Fig. 4.21 and Fig. 4.22 for illustrations). Note that we need to evaluate the objective function before we know whether a candidate point satisfies the sufficient decrease condition and becomes a good enough solution.

To find the value of α_k that meets (3.54), an initial guess of α_k is needed. Some methods, such as the Newton's method and BFGS algorithm, provide the initial guess, which is equal to 1. In many other cases, we normally start with a guess set to be the maximum step size and successively shrink it (typically with a factor of 0.5) until a good enough solution is found. This bisection-like line search method guarantees finding a good enough solution and is summarized in Algorithm 4.2 in the textbook.

The above approach has a few obvious drawbacks. It may require a large number of iterations to yield a solution if the initial guess is too large. Or it could lead to a conservative step size if the initial guess is too small (this becomes more severe if we increase the value of μ_1). The underlying reason is that we consider the objective function values only.

Strong Wolf conditions

We shall account for the gradient information during the line search to improve efficiency. The aim is to favor a large step size with reasonable decrease in the objective function, which could speed up the convergence and encourage exploring the design variable space. For this purpose, we include an extra condition, called the *sufficient curvature condition*, which is given by

$$|\phi'(\alpha_k)| \leq \mu_2 |\phi'(0)|. \quad (3.55)$$

In words, we further require that if a solution is a good enough one, the directional derivative at the corresponding step size should have a magnitude smaller than a fraction of the directional derivative at 0 (i.e., the function $\phi(\alpha_k)$ should be *flattening/shallow* here). See Fig. 4.25 for an illustration. The typical value for μ_2 is between 0.1 and 0.9. Setting μ_2 to be zero requires the line search to find a local minimum along the search direction $\mathbf{p}_k$. But we are not going to pursue this, in order to reduce the number of objective function evaluations.

To see why the sufficient curvature condition helps find a better step size, consider the case where $\phi'(\alpha_k) < \mu_2 \phi'(0)$. This means that at the current step size, the directional derivative has the same sign and a similar magnitude as $\phi'(0) < 0$, which indicates that the current step size is too small. On the other hand, if $\phi'(\alpha_k) > -\mu_2 \phi'(0)$ (i.e., the directional derivative is positive and has a similar magnitude as $\phi'(0)$), the current step size is too big.

The sufficient decrease condition and sufficient curvature condition are collectively known as the *strong Wolf conditions*. Fig. 4.26 in the textbook illustrates the region of step size values that satisfy the strong Wolf condition. Note that μ_2 should be much larger than μ_1 .

Bracketing and pinpointing

We present a line search technique that finds a step size satisfying the strong Wolf conditions. The method has two phases, namely the *bracketing* phase and the *pinpointing* phase. The bracketing phase aims at finding an interval within which it is guaranteed

to find a solution satisfying the strong Wolf conditions. The pinpointing phase locates a point within the interval provided by the bracketing phase.

The bracketing algorithm is summarized in [Algorithm 4.3](#) in the textbook. Essentially, it starts with an initial guess of the step size α_{init} . Then, we need to compute the objective function value $\phi(\alpha_{init})$ and its derivative $\phi'(\alpha_{init})$. If one of the following holds,

- $\phi(\alpha_{init}) > \phi(0)$,
- α_{init} satisfies the sufficient decrease condition and $\phi'(\alpha_{init}) > -\mu_2\phi'(0) > 0$,

Then, we have found the desired interval in the bracketing phase, which is $(0, \alpha_{init})$. If α_{init} satisfies the sufficient decrease condition and $|\phi'(\alpha_{init})| \leq \mu_2|\phi'(0)|$, we already locate a 'good enough' solution that meets the strong Wolf conditions, which is $\mathbf{x}_{k-1} + \alpha_{init} + \mathbf{p}_k$. If α_{init} satisfies the sufficient decrease condition but its gradient is even small than $\mu_2\phi'(0)$ (i.e., $\phi'(\alpha_{init}) < \mu_2\phi'(0)$), this indicates that we can enlarge the initial guess of the step size to find a point still satisfying the strong Wolf conditions but with a larger step size. In this case, we increase the initial guess by a factor of σ and repeat the above process until $(0, \sigma\alpha_{init})$ is the desired interval or $\sigma\alpha_{init}$ is a good solution already. See [Fig. 4.28](#) in the textbook for an illustration of those four cases.

Once an interval is found in the bracketing phase, a bisection search can be invoked to find a point that satisfies the strong Wolf conditions. A more complicated method but with faster convergence is detailed in [Algorithm 4.4](#) in the textbook.

3.3.5 Stopping criteria

The iterative techniques developed so far require some criteria to stop the iterations and output the obtained solution. According to [Chapter 4.2](#), for a solution $\mathbf{x}^*$ to be a local minimum, the gradient of the objective function at $\mathbf{x}^*$ should be zero. Thus, a possible stopping criterion is that the maximum (absolute) component of $\nabla f(\mathbf{x}^*)$ is less than a threshold τ , i.e.,

$$\|\nabla f(\mathbf{x}^*)\|_\infty \leq \tau, \quad (3.56)$$

where $\|\cdot\|_\infty$ is the infinity norm defined in [\(2.14\)](#). A typical value for τ is 10^{-6} . To take into account the magnitude of the gradient, a commonly used criterion is

$$\|\nabla f(\mathbf{x}^*)\|_\infty \leq \tau(1 + \|\nabla f(\mathbf{x}^0)\|_\infty), \quad (3.57)$$

where $\nabla f(\mathbf{x}_0)$ denotes the gradient at the initial solution guess $\mathbf{x}_0$.

Normally, it is not necessary to examine the Hessian $\mathbf{H}_f(\mathbf{x}^*)$ (see [Chapter 4.2.2](#)), because the considered iterative methods all utilize the search directions that lead to the reduction in the objective function.

In practical applications, the objective function may be poorly scaled, or there are other numerical issues that prevent the algorithm from satisfying [\(3.56\)](#) or [\(3.57\)](#). In these cases, to keep the algorithm from running indefinitely, a widely used criterion is to set a limit on the number of iterations or function calls. Another useful way is to examine the progress of the algorithm in terms of the amount of reduction in the objective function or the change in the solutions. Mathematically, we may stop the iteration if

$$|f(\mathbf{x}_{k+1}) - f(\mathbf{x}_k)| \leq \tau(1 + |f(\mathbf{x}_k)|); \quad (3.58a)$$

$$\|\mathbf{x}_{k+1} - \mathbf{x}_k\|_\infty \leq \tau(1 + \|\mathbf{x}_k\|_\infty). \quad (3.58b)$$

If you are applying a third-party optimizer, it is important to keep in mind the [Tip 4.1](#) on [Page 94](#) of the textbook.

3.3.6 Algorithm Initialization

The gradient-based algorithms presented in [Chapter 3.4](#) are all local search methods. If the objective function is multimodal, the quality of the local minimum found by the gradient-based algorithms may heavily depend on the initial solution guess $\mathbf{x}_0$.

A straightforward way to improve the performance of the gradient-based method is to augment them with a *multistart* approach, where multiple solution guesses are selected from engineering intuition (i.e., educated guesses on the basis of your understanding of the optimization problem in consideration), or sampling methods. By utilizing multiple initial solution guesses, we increase the chance of finding a good solution. If we converge to the same local minimum, even though we start with multiple well-spaced initial solutions, the objective function may not be multimodal at all.

The naive *full factorial* sampling, where we discretize each dimension of the input to the objective function and generate a grid containing all the combinations, is highly inefficient. This is because it scales exponentially with the number of design variables (i.e., it suffers from the *curse of dimensionality*). Thus, it is a common practice to identify the most influential design variables first, through e.g., sensitivity analysis.

Another straightforward way for sampling is random sampling, where the samples are obtained by following a desired distribution such as uniform distribution to draw independent samples in the design space. It scales better than the full factorial sampling but it is subject to potential clustering of the samples (see [Fig. 10.3](#) in the textbook for an example). We thus need a good sampling strategy to cover the design space efficiently than the full factorial sampling or random sampling.

We shall describe two commonly used sampling methods in the following.

Latin hypercube sampling (LHS)

The idea behind could be best illustrated by considering a 2D case (i.e., two design variables). Suppose we want only 8 samples. We can first generate a 8×8 grid by discretizing each dimension to 8 intervals. To cover the whole design space efficiently using only 8 samples, we can require that each row and each column of the grid has only one sample. Therefore, when projecting the samples along any dimension, the histogram should be uniform, indicating a good coverage. This concept is called the *Latin square* and its extension to higher dimensions is called the *Latin hypercube*.

Apparently, the Latin square or Latin hypercube is not unique (see [Fig. 10.6](#) in the textbook for two examples). One approach is to find the desired Latin square/hypercube through solving a constrained optimization problem. The problem requires that in any given row or column, there is only one sample and the minimum distance between any sample pair is maximized.

Solving the above problem may be challenging. A much simpler approach typically used in practice is given in [Algorithm 10.1](#) in the textbook. We shall illustrate a simplified version of it. Suppose that we would like to generate n_s samples in a design space of n_d dimensions. For simplicity, assume that the design variable in each dimension is bounded in the range $[0, 1]$. The algorithm first discretizes *randomly* each dimension using

$$V_{ij} = \frac{i}{n_s} - \frac{R_{ij}}{n_s}, \quad i = 1, 2, \dots, n_d, \quad j = 1, 2, \dots, n_s. \quad (3.59)$$

R_{ij} represents a sample independently drawn from the uniform distribution over $[0, 1]$. In words, it partitions each dimension into n_s uniform intervals and then draws a sample

from each interval. The samples V_{ij} is then collected in a $n_d \times n_s$ matrix, each row of which corresponds to a certain dimension and each column of which corresponds to a sample. Note that this step produces a Latin square/hypercube already but the samples are now distributed along the 'diagonal' of the design space. To improve the coverage of the design space, we randomly permute each row (again see Fig. 10.6 in the textbook for illustration). The above process can be repeated a couple of times and we select the Latin square/hypercube with the largest summed/minimum distance between points as the algorithm output.

The above algorithm can be easily generalized to the case where the projection of the samples along certain dimension needs to satisfy a distribution other than the uniform distribution. The only modification is to apply the *inversion sampling* technique to the samples V_{ij} along the particular dimension before random permutation. This can be considered as changing the histogram/intervals along that dimension to approximate the desired distribution. For more details on the inversion sampling technique, please refer to Page 397 and Fig. 10.7 of the textbook.

Halton and Hammersley sequences

Halton and Hammersley sequences both belong to the family of *low-discrepancy* sequences that are deterministic and well spatially spread. An attractive property is that the length of these sequences can be extended in an *online* manner. In other words, they need not to be generated beforehand and new points can be added whenever required. Moreover, each new point added to the sequence maintains low discrepancy. This property is particularly important for iterative procedures such as quadrature (i.e., numerical integration), Monte Carlo simulation, and multistart optimization algorithms (for further improving the quality of the solution). Here, discrepancy means the variation of the point density throughout the design space. Low discrepancy thus refers to even point density.

Halton and Hammersley sequences are generated based on the generalization of the one-dimensional *van der Corput sequence*. Let i be an integer. We represent i in base b as

$$i = a_0 + a_1b + a_2b^2 + \cdots + a_rb_r, \quad (3.60)$$

where $0 < a_j \leq b - 1$, $j = 0, 1, 2, \dots, r$. The integer i is thus a $(r + 1)$ -digit number in base b . When $b = 2$, we have the well-known binary representation.

Using (3.60), the i -th element $\phi_i(b)$ in the generalized van der Corput sequence is found via

$$\phi_b(i) = \frac{a_0}{b} + \frac{a_1}{b^2} + \cdots + \frac{a_r}{b^{r+1}}. \quad (3.61)$$

This method is known as the *radical inverse function* for base b , which is summarized in Algorithm 10.2 in the textbook. An example is given here. For base 2, the generated sequence is

$$\frac{1}{2}, \frac{1}{4}, \frac{3}{4}, \frac{1}{8}, \frac{5}{8}, \frac{3}{8}, \frac{7}{8}, \frac{1}{16}, \dots \quad (3.62)$$

In words, the interval $[0, 1]$ is split in half and then each subinterval is further halved.

Halton sequence for multiple dimensions are produced by using different prime numbers for each dimension. That is, the i -th Halton sequence is generated using

$$[\phi_{b_1}(i), \phi_{b_2}(i), \dots, \phi_{b_{n_d}}(i)]. \quad (3.63)$$

An example of a two-dimensional Halton sequence is given in Fig. 10.10 in the textbook.

Hammersley sequence can be considered as a special case of Halton sequence. It requires the number of points to be generated n_p is known beforehand. Another difference is that for the first dimension, regular spacing is used. The i -th Hammersley sequence is generated using

$$\left[\frac{i}{n_p}, \phi_{b_1}(i), \phi_{b_2}(i), \dots, \phi_{b_{n_d-1}}(i) \right]. \quad (3.64)$$

An example of a two-dimensional Hammersley sequence is given in Fig. 10.12 in the textbook.

3.4 Computing the Gradient/Derivatives

The gradient-based techniques developed in Chapter 4.3 for the unconstrained optimization problems all require the gradient of the objective function with respect to the design variables. Each element of the gradient is simply the partial derivative of the objective function with respect to a particular design variable. Evaluating the derivatives efficiently is central in the numerical optimization algorithms.

The methods for computing derivatives can be grouped into three categories, according to the amount of the information about the function. If we only have access to the inputs and outputs of a function (i.e., the function can be considered as a black box), its (partial) derivatives can be found using finite differences detailed below. When we have the analytical expression of the function or have access to every line of code for realizing the function of interest, we can find the derivatives through algebraic manipulations or perform algorithmic differentiation. In the intermediate case, implicit analytical methods are used. See Fig. 6.2 in the textbook for an illustration.

Finite differences

Finite-difference methods are simple but versatile. They require function values only for derivative evaluation. Many gradient-based optimization software perform finite differences by default when the user does not provide the required gradient.

Finite-difference methods compute each element of the gradient individually. Suppose we are interested in finding $\partial f(\mathbf{x})/\partial x_j$. For this purpose, we first approximate the function $f(\mathbf{x})$ along the j -th coordinate axis using the Taylor series expansion, which is

$$f(\mathbf{x} + h\mathbf{e}_j) = f(\mathbf{x}) + h \frac{\partial f(\mathbf{x})}{\partial x_j} + \frac{h^2}{2!} \frac{\partial^2 f(\mathbf{x})}{\partial x_j^2} + \frac{h^3}{3!} \frac{\partial^3 f(\mathbf{x})}{\partial x_j^3} + \dots \quad (3.65)$$

Here, $\mathbf{e}_j$ denotes the j -th column of a $n \times n$ identity matrix (i.e., its j -th element is 1 and other elements are all equal to zero). From (3.65), a straightforward way to approximate the desired partial derivative is given in the following *forward difference* formulae:

$$\frac{\partial f(\mathbf{x})}{\partial x_j} \approx \frac{f(\mathbf{x} + h\mathbf{e}_j) - f(\mathbf{x})}{h}, \quad (3.66)$$

where $h > 0$ is commonly called the *finite difference step size*. Again from (3.65), we have that (3.67) is a first-order approximation as the truncation error is $\mathcal{O}(h)$. Similarly, we can easily obtain another first-order approximation, which is the *backward difference* formulae:

$$\frac{\partial f(\mathbf{x})}{\partial x_j} \approx \frac{f(\mathbf{x}) - f(\mathbf{x} - h\mathbf{e}_j)}{h}. \quad (3.67)$$

The other elements in the required gradient can be computed in the same manner. Thus, the finite differences have a complexity proportional to the dimensionality of the inputs.

We can compute both the forward difference and backward difference, and average them to obtain an improved estimate of the derivative, which is

$$\frac{\partial f(\mathbf{x})}{\partial x_j} \approx \frac{f(\mathbf{x} + h\mathbf{e}_j) - f(\mathbf{x} - h\mathbf{e}_j)}{2h}. \quad (3.68)$$

It can be shown (Exercise 4.8) that this is a second-order estimate (i.e., the truncation error is $\mathcal{O}(h^2)$).

Note that according to (3.65), adding $f(\mathbf{x} + h\mathbf{e}_j)$ and $f(\mathbf{x} - h\mathbf{e}_j)$ cancels out the first-order and third-order derivatives. Besides, the second-order derivatives are kept and can be estimated using

$$\frac{\partial^2 f(\mathbf{x})}{\partial x_j^2} \approx \frac{f(\mathbf{x} + h\mathbf{e}_j) - 2f(\mathbf{x}) + f(\mathbf{x} - h\mathbf{e}_j)}{h^2}. \quad (3.69)$$

Finite difference methods are subject to the *step size dilemma*. It can be observed from the finite differences have a truncation error $\mathcal{O}(h)$ or $\mathcal{O}(h^2)$. This indicates that using a smaller finite difference step size h can improve the estimate. However, as h decreases, the *subtractive cancellation* effect, where the finite difference estimate can be entirely wrong, becomes more prominent. See Example 6.3 and Table 6.1 in the textbook for an example.

There exists an *optimal* step size. To find it, an iterative search is required. One way to search for a good step size is to start with a relatively large value of h (e.g., $h = 0.01$). Every time, the step size is decreased by a factor ρ (e.g., $\rho = 10$). Usually, the derivative estimate would monotonically decrease/increase at the beginning. The iteration terminates when the monotonic decrease/increase stops. In practice, to take into account the scaling of different design variables, we normally use a step size equal to $h(1 + |x_j|)$.

Complex step

The complex-step derivative approximation computes the partial derivatives using *complex* variables. The basic idea comes from noting that if we use a pure *imaginary* step size ih , the Taylor series expansion of the function of interest at $\mathbf{x}$ would become

$$f(\mathbf{x} + ih\mathbf{e}_j) = f(\mathbf{x}) + ih \frac{\partial f(\mathbf{x})}{\partial x_j} - \frac{h^2}{2!} \frac{\partial^2 f(\mathbf{x})}{\partial x_j^2} - i \frac{h^3}{3!} \frac{\partial^3 f(\mathbf{x})}{\partial x_j^3} + \dots \quad (3.70)$$

By taking the imaginary part on both sides, we arrive at

$$\frac{\partial f(\mathbf{x})}{\partial x_j} \approx \frac{\text{Im} f(\mathbf{x} + ih\mathbf{e}_j)}{h}, \quad (3.71)$$

with a truncation error $\mathcal{O}(h^2)$.

The significant advantage of this method is that the complex step method does not have subtraction, which eliminates the impact of subtractive cancellation. This enables the use of a very small value of the step size h , close to the machine precision (in fact, a reasonable choice is the square root of the smallest representable number), and improves the estimation accuracy greatly. Another byproduct of evaluating $f(\mathbf{x} + ih\mathbf{e}_j)$ is that we

can simultaneously estimate the function value at $\mathbf{x}$ with a truncation error $\mathcal{O}(h^2)$, which is given by

$$f(\mathbf{x}) \approx \operatorname{Re}\{f(\mathbf{x} + ih\mathbf{e}_j)\}. \quad (3.72)$$

Here, $\operatorname{Re}\{\cdot\}$ takes the real part.

The challenge of realizing the complex step method is that we must convert the code and the associated arithmetic for function evaluation into one that can handle complex numbers *correctly*. Programming languages such as Python and MATLAB are overloaded to automatically accept complex numbers. But necessary changes are needed if the program has logic operators ($\geq, \leq, \text{if}, \text{else}, \text{max}, \text{min}, \dots$). Normally, we must add the $\operatorname{Re}\{\cdot\}$ operations to make them correct. Besides, some functions may have to be replaced under complex arguments. For example, for the absolute value function, it does not output any imaginary part at all. A reasonable modified version is

$$\operatorname{abs}(x + iy) = \begin{cases} -x - iy, & \text{if } x < 0 \\ -x + iy, & \text{if } x \geq 0 \end{cases}. \quad (3.73)$$

Refer to <http://bit.ly/complexstep> for a list of problematic functions, implementation guide and scripts.

Chapter 4

Constrained Optimization

Many (most) engineering optimisation problems are subject to *constraints*. For example, any problem involving voltages and currents will have maximum voltages that the circuit can tolerate, maximum currents due to wire material and thickness, etc. Constraints may be physical limitations, or even equations which guarantee that the design does what it is supposed. E.g. if we are optimizing an electric vehicle design, we will likely have constraints on features we want, e.g. the range of the car should be at least 200 km, or 300 km, or whatever we want.

This chapter is a summarisation of Chapter 5 of the course textbook (Martin & Nings) and as such, text and equations are (at times) **copied verbatim without further attribution**. The author therefore claims no credit for the material in this chapter, only responsibility for any errors or mistakes!

4.1 Introduction

This chapter focuses on *constrained* optimization problems which can be expressed as

$$\begin{aligned} \min f(\mathbf{x}) \\ \text{subject to } \underline{x}_i \leq x_i \leq \bar{x}_i, i = 1, 2, \dots, n \\ g_j(\mathbf{x}) \leq 0, j = 1, 2, \dots, n_g \\ h_l(\mathbf{x}) = 0, l = 1, 2, \dots, n_h. \end{aligned} \tag{4.1}$$

where n_g denotes the number of inequality constraints while $n_h \leq n$ represents the number of equality constraints.

The optimality conditions for constrained optimization problems are not as straightforward as those for unconstrained optimization. We begin with equality constraints because the mathematics and intuition are simpler, then add inequality constraints.

Both equality and inequality constraints state a condition that must be satisfied in order for the problem to be solved. If it is not possible to satisfy these constraints, the optimisation problem is infeasible. Equality constraints can be thought of as constraining the solution to lie on a line or plane in n -space (in a problem with 2 design variables and one equality constraint, it is exactly a line; whereas if there are 3 decision variables and one equality constraint, the solution must lie on a plane). Inequality constraints define a region in which the solution must lie. Therefore, the optimal value of the objective function can never improve (i.e. decrease, since we have formulated a minimization problem) via adding constraints.

4.2 Equality constraints

Let us consider the problem (4.1) with $n, n_g = 0$ i.e. only equality constraints. From the previous section, recall that $\nabla f(\mathbf{x})^T \mathbf{p} \geq 0$ was a necessary condition for optimality, i.e. in all directions the objective function does not improve. Of course, a local minimum $\mathbf{x}$ as opposed to the global minimum $\mathbf{x}^*$ would satisfy this condition as well, so this is not a sufficient condition for optimality.

What happens if we have equality constraints? We still need $\nabla f(\mathbf{x}^*)^T \mathbf{p} \geq 0$ but now $\mathbf{p}$ is not every possible direction, but only *feasible* directions. Why? It does not matter if the objective function improves if we are barred from that region via an equality constraint.

How do we find feasible directions $\mathbf{p}$? We use the same Taylor-series approach:

$$h_j(\mathbf{x} + \mathbf{p}) \approx h_j(\mathbf{x}) + \nabla h_j(\mathbf{x})^T \mathbf{p}, l = 1, 2, \dots, n_h. \quad (4.2)$$

Assuming that $\mathbf{x}$ is a feasible point, then $h_j(\mathbf{x}) = 0$ for all constraints

$$h_j(\mathbf{x} + \mathbf{p}) \approx \nabla h_j(\mathbf{x})^T \mathbf{p}, l = 1, 2, \dots, n_h. \quad (4.3)$$

To remain feasible, $h_j(\mathbf{x} + \mathbf{p}) = 0$ for all j . We can write this in matrix form using the Jacobian matrix J_h as:

$$J_h(\mathbf{x})\mathbf{p} = 0. \quad (4.4)$$

This equation states that any feasible direction has to lie in the nullspace of the Jacobian of the constraints. What does this mean? Assuming that the constraints are linearly independent, J_h has full row rank and thus reduces the feasible space. In fact, for optimisation to be possible, $n > n_h$ (otherwise the feasible space reduces to a single point). See Fig. 5.8 in the textbook for illustration.

Having worked out our feasible directions, we now have that for $\mathbf{x}^*$ to be a constrained optimum

$$\nabla f(\mathbf{x}^*)^T \mathbf{p} \geq 0 \text{ for all } \mathbf{p} \text{ such that } J_h(\mathbf{x}^*)\mathbf{p} = 0. \quad (4.5)$$

Now, if $\mathbf{p}$ is a feasible direction, so is $-\mathbf{p}$ (think of a line - if we can go in one direction, we can also go in the other). Therefore, we require

$$\nabla f(\mathbf{x}^*)^T \mathbf{p} = 0 \text{ for all } \mathbf{p} \text{ such that } J_h(\mathbf{x}^*)\mathbf{p} = 0. \quad (4.6)$$

Now, we have two equations, $\nabla f(\mathbf{x}^*)^T \mathbf{p} = 0$ and $J_h(\mathbf{x}^*)\mathbf{p} = 0$ which obviously bear resemblance to each other. This has a mathematical interpretation. Mathematically, (4.2) shows that p is in the nullspace of J_h . The row space of a matrix contains all the vectors that are orthogonal to its nullspace, and the rows of J_h are gradients of the constraints. Thus, the objective function gradient must be a linear combination of the gradients of the constraints.

(4.2) can thus be written as:

$$\nabla f(\mathbf{x}^*) = - \sum_{j=1}^{n_h} \lambda_j \nabla h_j(\mathbf{x}^*) \quad (4.7)$$

where λ_j are called the *Lagrange multipliers*. In matrix notation, we therefore have:

$$\begin{aligned} \nabla f(\mathbf{x}) &= -J_h(\mathbf{x})\lambda \\ h(\mathbf{x}) &= 0 \end{aligned} \quad (4.8)$$

In constrained optimization, it is sometimes convenient to use the Lagrangian function, which is a scalar function defined as

$$\mathcal{L}(\mathbf{x}, \lambda) = f(\mathbf{x}) + h(\mathbf{x})^T \lambda. \quad (4.9)$$

Taking the gradient of $\mathcal{L}$ with respect to both $\mathbf{x}$ and λ and setting them to zero yields (4.8). We have therefore transformed an n dimension constrained optimisation problem to an $n + n_h$ dimension unconstrained problem. However, this is not usually a good way to solve constrained optimization problems.

The optimality conditions just described are first-order conditions that are necessary but not sufficient. To make sure that a point is a constrained minimum, we also need to satisfy second-order conditions. For the unconstrained case, the Hessian of the objective function has to be positive definite. This is also sufficient for the constrained case, but we can get a less conservative condition since we only need to check the Hessian of the Lagrangian in the space of feasible directions. We therefore require the Hessian of the Lagrangian function to be positive for any feasible direction p .

Do [Example 5.2](#) and [Example 5.3](#) in the textbook.

4.3 Inequality constraints

We will re-use many of the same concepts, but must recall some terminology to begin.

- A constraint is *active* if $g_j(\mathbf{x}^*) = 0$.
- A constraint is *inactive* if $g_j(\mathbf{x}^*) < 0$.
- A constraint is *feasible* when $g_j(\mathbf{x}^*) \leq 0$.

As before, if x^* is an optimum, any small enough feasible step $\mathbf{p}$ from the optimum must result in a function increase. Based on Taylor series expansion, we have the condition:

$$\nabla f(\mathbf{x}^*)^T \mathbf{p} \geq 0 \quad (4.10)$$

which is the same as for the equality constrained case.

To consider inequality constraints, we use the same linearization as the equality constraints (4.2), but now we enforce an inequality to get:

$$g_j(\mathbf{x} + \mathbf{p}) \approx g_j(\mathbf{x}) + \nabla g_j(\mathbf{x})^T \mathbf{p} \leq 0, j = 1, 2, \dots, n_g. \quad (4.11)$$

For a given candidate point that satisfies all constraints, there are two possibilities to consider for each inequality constraint: whether the constraint is inactive ($g_j(\mathbf{x}^*) < 0$) or active ($g_j(\mathbf{x}^*) = 0$). If a given constraint is inactive, we do not need to add any condition for it because we can take a step $\mathbf{p}$ in any direction and remain feasible as long as the step is small enough. Thus, we only need to consider the active constraints for the optimality conditions.

For the equality constraint, we found that all first-order feasible directions are in the nullspace of the Jacobian matrix. Inequality constraints are not as restrictive. From (4.3), if constraint j is active ($g_j(\mathbf{x}) = 0$), then the nearby point $g_j(\mathbf{x} + \mathbf{p})$ is only feasible if $\nabla g_j(\mathbf{x})^T \mathbf{p} \leq 0$ for all constraints j that are active. In matrix form, we can write $J_g(\mathbf{x})\mathbf{p} \leq 0$, where the Jacobian matrix includes only the gradients of the active constraints.

The set of feasible directions that satisfies all active constraints is the intersection of all the closed half-spaces defined by the inequality constraints, that is, all $\mathbf{p}$ such that $J_g(\mathbf{x})\mathbf{p} \leq 0$.

This intersection of the feasible directions forms a polyhedral cone.

To find the cone of feasible directions, let us first consider the cone formed by the active inequality constraint gradients. This cone is defined by all vectors $\mathbf{d}$ such that:

$$\mathbf{d} = J_g^T \sigma = \sum_{j=1}^{n_g} \sigma_j \nabla g_j, \text{ where } \sigma_j \geq 0. \quad (4.12)$$

A direction $\mathbf{p}$ is feasible if $\mathbf{p}^T \mathbf{d} \leq 0$ for all $\mathbf{d}$ in the cone. The set of all feasible directions forms the polar cone of the cone defined by (4.3).

4.3.1 Farkas' Lemma

Now that we have established some intuition about the feasible directions, we need to establish under which conditions there is no feasible descent direction (i.e., we have reached an optimum). In other words, when is there no intersection between the cone of feasible directions and the open half-space of descent directions? To answer this question, we can use Farkas' lemma. This lemma states that given a rectangular matrix A with size $m \times n$ and a vector $\mathbf{b}$ with length m (i.e. the same length as the rows of the matrix), one (and only one) of two possibilities occurs:

1. There exists an $m \times 1$ vector $\mathbf{y}$ such that $A^T \mathbf{y} \geq 0$ and $\mathbf{b}^T \mathbf{y} < 0$.
2. There exists an $n \times 1$ vector $\mathbf{x}$ such that $A\mathbf{x} = \mathbf{b}$ and $\mathbf{x} \geq 0$.

In our case, replace A by the rectangular (i.e. $m = n$) matrix J_g^T , $\mathbf{b}$ by $-\nabla f$, $\mathbf{x}$ by σ , and $\mathbf{y}$ by $-\mathbf{p}$, and we have the following two possibilities:

1. There exists an $n \times 1$ vector $\mathbf{p}$ such that $J_g \mathbf{p} \leq 0$ and $\nabla f^T \mathbf{p} < 0$.
2. There exists an $n \times 1$ vector σ such that $J_g^T \sigma = \nabla f$ and $\sigma \geq 0$.

In the first case, this means there is a feasible direction $J_g(\mathbf{x}^*)\mathbf{p} \leq 0$ for which we can descend $\nabla f(\mathbf{x}^*)^T \mathbf{p} < 0$. In the second case, we can rearrange to state our optimality criterion:

$$\nabla f(\mathbf{x}^*) + J_g(\mathbf{x}^*)^T \sigma = 0, \text{ with } \sigma \geq 0. \quad (4.13)$$

This is similar to the conditions (4.8) for equality constraints. σ corresponds to the Lagrange multipliers for the inequality constraints and carries the additional restriction that $\sigma \geq 0$.

4.3.2 Slackness

Similar to the equality constrained case, we can construct a Lagrangian function whose stationary points are candidates for optimal points. We need to include all inequality constraints in the optimality conditions because we do not know in advance which constraints are active. To represent inequality constraints in the Lagrangian, we replace them with the equality constraints defined by:

$$g_j + s_j^2 = 0, j = 1, \dots, n_g. \quad (4.14)$$

where s_j is a new unknown associated with each inequality constraint called a *slack variable*. The slack variable is squared to ensure it is nonnegative, i.e. if (4.14) then $g_j \leq 0$ and the inequality constraint j is feasible.

The Lagrangian including both equality and inequality constraints is then

$$\mathcal{L}(\mathbf{x}, \lambda, \sigma, \mathbf{s}) = f(\mathbf{x}) + \lambda^T h(\mathbf{x}) + \sigma^T (g(\mathbf{x}) + \mathbf{s} \odot \mathbf{s}) \quad (4.15)$$

where σ represents the Lagrange multipliers associated with the inequality constraints. Here, we use $\odot$ to represent the element-wise multiplication of $\mathbf{s}$.

4.4 KKT conditions

Similar to the equality constrained case, we seek a stationary point for the Lagrangian, but now we have additional unknowns: the inequality Lagrange multipliers and the slack variables. Taking partial derivatives of the Lagrangian with respect to each set of unknowns and setting those derivatives to zero yields the first-order optimality conditions:

$$\nabla_x \mathcal{L} = 0 \implies \frac{\partial \mathcal{L}}{\partial x_i} = \frac{\partial f}{\partial x_i} + \sum_{l=1}^{n_h} \lambda_l \frac{\partial h_l}{\partial x_i} + \sum_{j=1}^{n_g} \sigma_j \frac{\partial g_j}{\partial x_i} = 0, i = 1, \dots, n_x. \quad (4.16)$$

This criterion is the same as before but with additional Lagrange multipliers and constraints. Taking the derivatives with respect to the equality Lagrange multipliers, we have:

$$\nabla_\lambda \mathcal{L} = 0 \implies \frac{\partial \mathcal{L}}{\partial \lambda_l} = h_l = 0, l = 1, \dots, n_h. \quad (4.17)$$

which enforces the equality constraints as before. Taking derivatives with respect to the inequality Lagrange multipliers, we get:

$$\nabla_\sigma \mathcal{L} = 0 \implies \frac{\partial \mathcal{L}}{\partial \sigma_j} = g_j + s_j^2 = 0, j = 1, \dots, n_g. \quad (4.18)$$

which enforces the inequality constraints. Finally, differentiating the Lagrangian with respect to the slack variables, we obtain:

$$\nabla_s \mathcal{L} = 0 \implies \frac{\partial \mathcal{L}}{\partial s_j} = 2\sigma_j s_j = 0, j = 1, \dots, n_g. \quad (4.19)$$

which is called the complementary slackness condition. This condition helps us to distinguish the active constraints from the inactive ones. For each inequality constraint, either the Lagrange multiplier is zero (which means that the constraint is inactive), or the slack variable is zero (which means that the constraint is active).

Unfortunately, the complementary slackness condition introduces a combinatorial problem. The complexity of this problem grows exponentially with the number of inequality constraints because the number of possible combinations of active versus inactive constraints is 2^{n_g} .

In addition to the conditions for a stationary point of the Lagrangian (4.16)-(4.19), recall that we require the Lagrange multipliers for the active constraints to be nonnegative. Putting all these conditions together in matrix form, the first-order constrained

optimality conditions are as follows:

$$\nabla_f + J_h^T \lambda + J_g^T \sigma = 0 \quad (4.20a)$$

$$h = 0 \quad (4.20b)$$

$$g + s \odot s = 0 \quad (4.20c)$$

$$\sigma \odot s = 0 \quad (4.20d)$$

$$\sigma \geq 0 \quad (4.20e)$$

As in the equality constrained case, these first-order conditions are necessary but not sufficient. The second-order sufficient conditions require that the Hessian of the Lagrangian must be positive definite in all feasible directions, that is,

$$\mathbf{p}^T H_\lambda \mathbf{p} > 0 \quad (4.21a)$$

$$J_h \mathbf{p} = 0 \quad (4.21b)$$

$$J_g \mathbf{p} \leq 0 \quad (4.21c)$$

In other words, we only require positive definiteness in the intersection of the nullspace of the equality constraint Jacobian with the feasibility cone of the active inequality constraints.

Do [Example 5.4](#) and [Example 5.5](#) in the textbook.

Although these examples can be solved analytically, this is often difficult or impractical. The KKT conditions quickly become challenging to solve analytically, and as the number of constraints increases, trying all combinations of active and inactive constraints becomes intractable. Furthermore, engineering problems usually involve functions defined by models with implicit equations, which are impossible to solve analytically. The reason we include these analytic examples is to gain a better understanding of the KKT conditions. For the rest of the section, we focus on numerical methods, which are necessary for the vast majority of practical problems.

4.5 Lagrange multipliers again

The Lagrange multipliers quantify how much the corresponding constraints drive the design. More specifically, a Lagrange multiplier quantifies the sensitivity of the optimal objective function value $f(\mathbf{x}^*)$ to a variation in the value of the corresponding constraint. Here we explain why that is the case. We discuss only inequality constraints, but the same analysis applies to equality constraints.

When a constraint is inactive, the corresponding Lagrange multiplier is zero. This indicates that changing the value of an inactive constraint does not affect the optimum, as expected. This is only valid to the first order because the KKT conditions are based on the linearization of the objective and constraint functions. Because small changes are assumed in the linearization, we do not consider the case where an inactive constraint becomes active after perturbation.

It can be derived (see [Section 5.3.3](#) of the textbook) that:

$$\sigma_i = -\frac{df}{dg_i} \quad (4.22)$$

Thus, the Lagrange multipliers can predict how much improvement can be expected if a given constraint is relaxed. For inequality constraints, because the Lagrange multipliers

are positive at an optimum, this equation correctly predicts a decrease in the objective function value when the constraint value is increased.

The derivative defined in (4.22) has practical value because it tells us how much a given constraint drives the design. In this interpretation of the Lagrange multipliers, we need to consider the scaling of the problem and the units. Still, for similar quantities, they quantify the relative importance of the constraints.

4.6 Algorithms for solving constrained optimization problems

In general, if we can solve the KKT conditions, we have solved the optimization problem. However, as mentioned previous, there is a combinatorial issue with directly solving the KKT conditions for many problems, so we need an algorithm.

There are several methods used to solve constrained optimisation problems, however the most important solvers are the *interior point* and *SQP* methods. We do not have time to describe any method in detail, but will give a quick introduction here to interior point methods. We start by discussing the fundamental concept of penalty methods which underpin interior point solvers.

4.6.1 Penalty methods

The concept behind penalty methods is intuitive: to transform a constrained problem into an unconstrained one by adding a penalty to the objective function when constraints are violated or close to being violated. As mentioned in the introduction to this chapter, penalty methods are no longer used directly in gradient-based optimization algorithms because they have difficulty converging to the true solution. However, these methods are still valuable because, among other reasons, (1) they are simple and thus ease the transition into understanding constrained optimization; (2) penalty concepts are used in interior points methods. The penalized function can be written as:

$$\hat{f}(\mathbf{x}) = f(\mathbf{x}) + \mu\pi(\mathbf{x}) \quad (4.23)$$

where $\pi(\mathbf{x})$ is a penalty function, and the scalar μ is a penalty parameter. This is similar in form to the Lagrangian, but one difference is that μ is fixed instead of being a variable.

Various forms for $\pi(x)$ can be used, leading to different penalty methods. There are two main types of penalty functions: *exterior* penalties, which impose a penalty only when constraints are violated, and *interior* penalty functions, which impose a penalty that increases as a constraint is approached. The exterior penalty leads to slightly infeasible solutions, whereas an interior penalty leads to a feasible solution but underpredicts the objective.

We can use the unconstrained optimization techniques to minimize $\hat{f}(x)$. However, instead of just solving a single optimization problem, penalty methods usually solve a sequence of problems with different values of μ to get closer to the actual constrained minimum. The value of the penalty parameter μ must be chosen carefully. Mathematically, we recover the exact solution to the constrained problem only as μ tends to infinity. However, starting with a large value for μ is not practical. This is because the larger the value of μ the larger the Hessian condition number, which corresponds to the curvature

varying greatly with direction. This behavior makes the problem difficult to solve numerically. To solve the problem more effectively, we begin with a small value of μ and solve the unconstrained problem. We then increase μ and solve the new unconstrained problem, using the previous solution as the starting point. We repeat this process until the optimality conditions (or some other approximate convergence criteria) are satisfied.

By gradually increasing μ and reusing the solution from the previous problem, we avoid some of the ill-conditioning issues. Thus, the original constrained problem is transformed into a sequence of unconstrained optimization problems.

4.6.2 Interior point solvers

These methods form an objective similar to the interior penalty but with the key difference that instead of penalizing the constraints directly, they add slack variables to the set of optimization variables and penalize the slack variables. The resulting formulation is:

$$\begin{aligned} \min_{\mathbf{x}, s} \quad & f(\mathbf{x}) - \mu_b \sum_{j=1}^{n_g} \log s_j \\ \text{subject to} \quad & g(\mathbf{x}) + s = 0 \\ & h(\mathbf{x}) = 0 \end{aligned} \tag{4.24}$$

Instead of solving the KKT conditions of the original problem, we solve the KKT conditions of the interior-point formulation (4.24). This results in a linear problem that can be easily solved (see Equation 5.100 in the textbook).

These slack variables in (4.24) do not need to be squared, as was done in deriving the KKT conditions, because the logarithm is only defined for positive s values and acts as a barrier preventing negative values of s .

Similarly to penalty methods, we have to move slowly to prevent issues with conditioning due to the gradients. Determining the correct step-size is a crucial part of any interior-point algorithm, and it should be noted that this is used to enforce $s > 0$ implicitly. Hence, at each iteration, the parameters (also the barrier coefficient μ_b) are updated until we converge.

Time prevents a discussion of convergence, or indeed further discussion of interior-point methods, but the textbook has an excellent and detailed description for those wanting to know more. See especially Algorithm 5.7 in the textbook.

Interior point solvers are common in practice and are implemented in multiple software packages, e.g. *fmincon* in MATLAB or SciPy, IPOPT, Knitro, etc. Note: for those interested to make the comparison with SQP, SQP methods are also available in MATLAB (*sqp* option in *fmincon*), SNOPT, Knitro, and other packages as well. Reading <https://au.mathworks.com/help/optim/ug/choosing-the-algorithm.html> may be helpful.

Chapter 5

Design Projects

5.1 Suggested Project 1: Power system optimization

Brief highlights

- Solve a relatively realistic power systems optimisation problem.
- Compete with your classmates for the best solution (bonus marks awarded).
- MATLAB or Python software used; evaluation and (sub-optimal) solution code provided as a starting point.
- High flexibility: a wide range of possible approaches are possible.

5.1.1 Introduction

In this assignment, your task will be to design a function to optimize the build and operation of generation (and storage if so desired) for a given power system. The aim is to maximize cash (i.e. profits plus any starting cash left unused; excluding asset value) at the end of a year-long simulation.

In the build stage, your programme will be given details of the system and parameters for each type of generation and decide what to build. In the operation stage, your second module will decide how much generation (of each type) to supply to the system for each hourly time period.

MATLAB and Python code to run the year-long simulation is provided on Learn, along with an example (sub-optimal) solution. Either language is fully acceptable and will result in equivalent performance in the final evaluation. Those exceedingly short of time have the option only to optimize the build process while using the given operation code. Such solutions should not expect the very highest marks, but a good result is still possible.

The following sections explain the problem in detail. However, please note that the code and associated comments may explain the problem more clearly than any written text. It is also **not** necessary to understand the simulation detail below - one could modify the provided code into an objective function and optimize the any/all parameters by any appropriate method without using intuition or other knowledge.

Generation	Build cost (\$ millions / MW)	Capacity factor
Wind	$\mathcal{N}(1.1, 0.11^2)$	Uniform $\in [0 \text{ } 50\%]$
Geothermal	$\mathcal{N}(4.8 \text{ } 0.48^2)$	100%
Hydro	$\mathcal{N}(2.8, 0.28^2)$	$\mathcal{N}(45\%, 4.5\%^2)$
Solar	$\mathcal{N}(0.8, 0.08^2)$	Average 24.58%

Table 5.1: Generation options

5.1.2 Notation

We will denote a Gaussian random variable X with mean μ and standard deviation σ by $X \sim \mathcal{N}(\mu, \sigma^2)$.

5.1.3 Problem formulation

Generation options

Four generation options are provided:

1. **Wind** is modelled by a random wind capacity factor from 0.00 to 0.50, i.e. the available wind power ranges from 0 to 50% of built capacity (uniform distribution). This value changes at each time period.
2. **Hydro** is fully controllable up to its rated power. However, the overall capacity factor throughout the year is limited to a constant value which is set for each year-long simulation via a Gaussian random variable with 0.45 mean and 0.045 standard deviation. When the cumulative total of hydro exports reaches this number, it is assumed that the hydro generator is out of water and no further generation is permitted. For simplicity, in-flows and out-flows etc. are not explicitly modelled.
3. **Geothermal** is fully controllable up to its rated power.
4. **Solar** is modelled by a 24-hour cycle capacity factor from 0.00 to 0.80 with 20% multiplicative randomness at each time period (uniform distribution). Full details are given in the provided code.

Table 5.1 shows the parameters for each generation option. Each Gaussian random variable has 10% standard deviation. Each uniform random variable is given bounds.

Storage

Battery storage is available with the following parameters:

- The cost is \$150,000 per MWh. This is randomized for each year-long simulation via a Gaussian random variable with 10% standard deviation.
- It is assumed that the rated power in MW is equal to the rated capacity in MWh, i.e. a charge / discharge time of one hour.
- One-way efficiency is 97.5%.
- The usable state-of-charge is from 0% to 100%.
- Batteries are fully charged at the start of the simulation.

Load

The system mean load L_{avg} is chosen randomly for each year-long simulation, uniformly distributed between [1000,2000] MW. For each time-period, the load at that time period is a Gaussian random variable with the mean as chosen above and a standard deviation of 500 MW, i.e. $\text{load}(t) \sim \mathcal{N}(L_{avg}, 500^2)$ MW.

Pricing

The market price will be set by the following formula at each time period t :

```
base_price(t) = base_price_average + base_price_average/10*randn
price(t) = base_price(t) + load(t)*priceCoefficient1
           - wind_capacity_factor(t)*priceCoefficient2
```

where $\text{base_price_average} \sim \mathcal{N}(150, 15^2)$, $\text{priceCoefficient1} \sim \mathcal{N}(0.075, 0.0075^2)$, and $\text{priceCoefficient2} \sim \mathcal{N}(150, 15^2)$ are chosen randomly for each year-long simulation. This price (in \$ per MWh) will be multiplied by the amount (in MWh) you decide to generate, and the result added to your funds.

If your programme does not return sufficient generation, the interface will penalize this as follows:

- A shortfall of up to $\text{external_MWs}(t)$ will be purchased at market price.
- Any further shortfall cannot be supplied and a penalty cost of \$1000 per MWh will be deducted from your funds.

If your programme returns excess generation, this is curtailed / spilled for free.

5.1.4 Solution methodology

Possible approaches include gradient-based methods, particle swarm optimization, neural networks, etc. Computational time may be a limitation, and it will be imperative to explain your approach and design choices clearly.

A solution has two parts:

1. A module `build_plants()` to decide how much of each type of power plant to build at the start of each year-long simulation. The interface will check if the proposed build is feasible, and throw an error if not.
2. A module `operate_plants()` to decide how much generation of each type to offer at each time-step. For solar, wind and geothermal generation, this is trivial as there is no benefit to reducing output (since curtailment is free). However, it is non-trivial to work out when battery storage should charge or discharge, and (given the finite amount of hydro energy) when to use hydro to avoid shortfalls.

Only a few constraints are placed upon your solution:

- No forecast or future information will be given, i.e. at a time period T , information from a future time-step $t > T$ will not be known.

- `operate_plants()` should execute in less than 1ms on a standard computer in the computer lab. If your code is close to this (this should not be an issue in most cases), please time your code and include the info in your report. You may use any method, e.g. (<https://au.mathworks.com/help/matlab/ref/timeit.html>) or the Profiler tool in MATLAB, or *profile* or *cProfile* in Python.
- `build_plants()` should execute in less than 1s on a standard computer in the computer lab. If your code is close to this (this should not be an issue in most cases), please time your code and include the info in your report. You may use any method as per the above.

In order to get you started, a demonstration solution has been provided:

- `build_plants()` spends 20% of its money building each of wind, geothermal, hydro, solar, and battery storage, regardless of given costs and other parameters. This is far from optimal.
- `operate_plants()` uses simple logic to avoid a shortfall whenever possible. Wind, solar and geothermal generations are run at their maximum available capacity at that point in time. If there is still a shortfall, any hydro generation is used to cover the shortfall, and if there is still a shortfall, any stored energy in the battery is used to reduce or (if possible) avoid the shortfall. There is also considerable room for improvement in this module.

Hints

1. Not all the starting funds need to be used building generation and/or storage. Since the objective is to maximize cash at the end of one year, it may be beneficial to under-invest. Any unused starting funds will be added to your cash flow at the end to make your final score.
2. Because there are penalties for failing to supply load (as in the real world), it may be beneficial to have a mix of generation and/or storage ...
3. The ideal mix of generation and/or storage is likely to depend on the loading, costs, and capacity factors which are randomized each year-long simulation. You may wish to adjust your build plans based on this info which is passed to `build_plants()`, as opposed to sticking to fixed proportions per the demo solution.
4. As is often the case for power system optimization problems, the global optimization problem may be computationally impractical to solve. Firstly, there are a number of stochastic (random) variables, the exact value of which is not known in advance. Secondly, there are 8760 time periods, and for each time period, generation + storage MW contributions are design variables (i.e. they need to be optimized), potentially resulting in over 40,000 design variables and even more constraints. Hence, it may be helpful to look for ways to simplify or decompose the problem.

5.1.5 Deliverables

The progress report should cover the following: a little background / intuition about the problem; a short overview of possible methods to use for the problem; a plan for

Item	Weight
Presentation	0.25
Quality of the technical design	0.50
Performance	0.25

Table 5.2: Final report marking

your solution; any progress-to-date. It is very easy to find a solution, even by trial and error, better than the given starting solution, so it is expected to show some preliminary improvement by the time of the progress report.

The final report should explain your solution in-depth (see below), and be accompanied by the MATLAB or Python code used to implement your solution. Please email this to me at jeremy.watson@canterbury.ac.nz or upload to the Learn dropbox. Your solution will be tested in the given environment and (per the next section) some marks will be awarded for performance.

5.1.6 Marking

The final report will be assessed as in Table 5.2:

Guidelines

Please take note of the following:

Presentation: Reports should be tidy and concise. Use graphs / figures as appropriate to illustrate your solution. Any format is acceptable, although IEEE conference format (<https://www.ieee.org/conferences/publishing/templates.html>) is suggested. It is important to focus on the parts you developed, as opposed to explaining the problem and/or provided code.

Quality of the technical design: Explain what methods you tried and justify your final choice. Solutions which attempt to optimize both the build and operation process are encouraged, and should demonstrate benefit compared to only optimizing the build using the provided operation code. Comparisons of different optimisation approaches are encouraged.

Performance: you should test your code with the provided evaluation function (on LEARN) and report its score in your final report. I will independently evaluate your solution by changing the rng seed and running the simulation with `totalSimulations` equal to 100, from which the average score will be taken. Marks will be awarded as follows:

1. The best solution from the class will score full marks for this section, so long as it achieves a score of \$6.5 billion or better.
2. Marks thereafter will be awarded based on the average score, using linear interpolation where the best score is 100 % and the class median score is set at 70%. However, regardless of this calculation, no negative marks will be awarded; all complete and working solutions better than the starting solution will score at least 30% for this section.

3. Bonus marks of 10% (of the marks assigned to performance) will be awarded for any solution which outperforms my own sub-optimal solution. If this results in a mark exceeding 100%, the mark will not be capped.

5.2 Suggested Project 2: Source Geolocation using Frequency Measurements

Project Summary

- Formulate the source geolocation problem under Gaussian measurement noise as a nonlinear least squares (NLS) problem.
- Derive the analytical expression for the gradient of the cost function and validate your results using numerical methods for computing the derivatives.
- Design and realize your own gradient-based algorithm with line search that takes into account the bound constraints on the design variables.
- Evaluate the geolocation accuracy of your method under different noise levels and compare it with at least one benchmark method at your choice.
- Analyze the efficiency of your method in terms of the number of iterations required by your gradient-based algorithm and the number of objective function evaluations required by your line search method.

5.2.1 Problem Description

Geolocation of radio sources on the Earth surface is a classic estimation problem, which has found numerous civil and military applications in navigation, search and rescue, and surveillance.

In this project, we shall consider the use of four satellites to locate a ground radio source using the frequency difference of arrival (FDOA) measurements only. This is known in some literature as FDOA-only localization, which is a recently exploited topic. For example, see the paper below and references there in:

[1]. Y. Pei, X. Li, Le Yang and F. Guo, “A closed-form solution for source localization using FDOA measurements only,” *IEEE Communications Letters*, vol. 27, pp. 115-119, Jan. 2022.

5.2.2 Mathematical Background

In FDOA-only geolocation, each satellite in the satellite group receives the signal of the *same* radio source. The source is located at $\mathbf{u}^o = [x^o, y^o, z^o]^T$ in the Earth Centered Earth Fixed (ECEF) coordinate system. Its geodetic position is denoted as $\mathbf{p}^o = [B^o, L^o, H^o]^T$, where B^o , L^o and H^o represents the latitude, longitude and altitude. Since the radio source is assumed to be on the ground, we have $H^o = 0$.

The Cartesian coordinates $\mathbf{u}^o$ are related to $\mathbf{p}^o$ via

$$x^o = (R_E(B^o) + H)\cos(B^o)\cos(L^o), \quad (5.1a)$$

$$y^o = (R_E(B^o) + H)\cos(B^o)\sin(L^o), \quad (5.1b)$$

$$z^o = [(1 - e^2)R_E(B^o) + H]\sin(B^o), \quad (5.1c)$$

where

$$R_E(B^o) = \frac{R_0}{\sqrt{1 - e^2 \sin^2(B^o)}}, \quad (5.2)$$

$e = 0.081819198425$ is the eccentricity, and $R_0 = 6378.137\text{km}$ is the Earth equatorial radius. For MATLAB functions on coordinate conversion, see

- <https://www.mathworks.com/help/map/ref/geodetic2ecef.html>,
- <https://www.mathworks.com/help/map/ref/ecef2geodetic.html>.

Let us denote the position and velocity of satellite i , $i = 1, 2, 3, 4$, in the ECEF coordinate system as $\mathbf{s}_i = [s_{x,i}, s_{y,i}, s_{z,i}]^T$ and $\dot{\mathbf{s}}_i = [\dot{s}_{x,i}, \dot{s}_{y,i}, \dot{s}_{z,i}]^T$. Suppose the radio source is transmitting with an unknown frequency f_0 . The signal frequency measured at satellite i is thus subject to the Doppler effect and is equal to

$$f_i = f_0 + \frac{1}{\lambda} \cdot \frac{(\mathbf{u}^o - \mathbf{s}_i)^T \dot{\mathbf{s}}_i}{\|\mathbf{u}^o - \mathbf{s}_i\|} + n_i, \quad (5.3)$$

where $\lambda = \frac{c}{f_0}$ is the source signal wavelength, and n_i is the Gaussian random noise with zero mean and variance σ_f (i.e., $n_i \sim \mathcal{N}(0, \sigma_f^2)$).

Since the source signal frequency is not known, we subtract the received frequency at satellite 1 (also called the reference satellite) from the received frequencies at other satellites to obtain the FDOA measurements¹. Specifically, the measured FDOA f_{j1} between the satellite pair j and 1, $j = 2, 3, 4$, is

$$f_{j1} = f_j - f_1 = \frac{1}{\lambda} \cdot \frac{(\mathbf{u}^o - \mathbf{s}_j)^T \dot{\mathbf{s}}_j}{\|\mathbf{u}^o - \mathbf{s}_j\|} - \frac{1}{\lambda} \cdot \frac{(\mathbf{u}^o - \mathbf{s}_1)^T \dot{\mathbf{s}}_1}{\|\mathbf{u}^o - \mathbf{s}_1\|} + (n_j - n_1). \quad (5.4)$$

Collecting f_{j1} for $j = 2, 3, 4$ generates the measurement vector $\mathbf{f} = [f_{21}, f_{31}, f_{41}]^T$. Besides, define $\mathbf{g}(B^o, L^o)$ as the true FDOA function

$$\mathbf{g}(B^o, L^o) = \begin{bmatrix} g(B^o, L^o, \mathbf{s}_2, \dot{\mathbf{s}}_2) - g(B^o, L^o, \mathbf{s}_1, \dot{\mathbf{s}}_1) \\ g(B^o, L^o, \mathbf{s}_3, \dot{\mathbf{s}}_3) - g(B^o, L^o, \mathbf{s}_1, \dot{\mathbf{s}}_1) \\ g(B^o, L^o, \mathbf{s}_4, \dot{\mathbf{s}}_4) - g(B^o, L^o, \mathbf{s}_1, \dot{\mathbf{s}}_1) \end{bmatrix}, \quad (5.5)$$

where from (5.4),

$$g(B^o, L^o, \mathbf{s}_i, \dot{\mathbf{s}}_i) = \frac{1}{\lambda} \cdot \frac{(\mathbf{u}^o - \mathbf{s}_i)^T \dot{\mathbf{s}}_i}{\|\mathbf{u}^o - \mathbf{s}_i\|}. \quad (5.6)$$

The measurement noise in $\mathbf{f}$ is thus equal to $\mathbf{n}_f$, which is

$$\mathbf{n}_f = \begin{bmatrix} n_2 - n_1 \\ n_3 - n_1 \\ n_4 - n_1 \end{bmatrix}. \quad (5.7)$$

We are interested in identifying the source position $\mathbf{p}^o$ (with its altitude $H^o = 0$) using the measured FDOAs in $\mathbf{f}$ as well as the known satellite positions $\mathbf{s}_i$ and velocities $\dot{\mathbf{s}}_i$, $i = 1, 2, 3, 4$.

¹This is how the name ‘FDOA’ comes from. We are using indeed the *difference* of the Doppler frequencies received at various satellites for source geolocation.

5.2.3 Guidelines for Progress Report

1). (30%) Show that the FDOA measurement vector $\mathbf{f}$ follows a multivariate Gaussian distribution with mean $\mathbf{g}(B^\circ, L^\circ)$ and covariance $\sigma_f^2 \mathbf{Q}$, where $\mathbf{Q}$ takes the form

$$\mathbf{Q} = \begin{bmatrix} 2 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 2 \end{bmatrix}. \quad (5.8)$$

Then, formulate the FDOA-only geolocation problem using the maximum likelihood (ML) estimation principle as a nonlinear least-squares problem.

Hint: Refer to [Chapter 7](#) in this notes for an introduction to the multivariate Gaussian density and ML estimation.

2). (40%) Consider the following scenario. The radio source has a carrier frequency of $f_0 = 1\text{GHz}$. It is located at $\mathbf{p}^\circ = [5^\circ, 10^\circ, 0]^T$, where 5° means the north latitude of 5 degrees and 10° means the east longitude of 10 degrees. The satellite positions and velocities are listed below:

$$\begin{aligned} \mathbf{s}_1 &= [7378.1, \quad 0, \quad 0]^T \text{ km}, \dot{\mathbf{s}}_1 = [0.0001, 4.4995, 5.3623]^T \text{ km/s}, \\ \mathbf{s}_2 &= [7377.5, \quad 100, \quad 0]^T \text{ km}, \dot{\mathbf{s}}_2 = [-0.0671, 4.9493, 4.9497]^T \text{ km/s}, \\ \mathbf{s}_3 &= [7377.5, -100, \quad 0]^T \text{ km}, \dot{\mathbf{s}}_3 = [0.0610, 4.4991, 5.3623]^T \text{ km/s}, \\ \mathbf{s}_4 &= [7377.5, \quad 0, \quad 100]^T \text{ km}, \dot{\mathbf{s}}_4 = [-0.0777, 4.0150, 5.7335]^T \text{ km/s}. \end{aligned}$$

It is priorly known that the latitude and longitude of the radio source are within $[0, 40^\circ]$ and $[0, 40^\circ]$. Implement the grid search method for accomplishing geolocation. The grid search process is as follows. First, the priorly known location area of the radio source is covered with a uniform grid. We then evaluate the objective function at each grid point. The grid point with the minimum objective function value is output as the geolocation result.

In the progress report, assume that the signal frequency of the radio source is *known*.

3). (30%) Implement a Monte Carlo simulation program to investigate the geolocation accuracy of your grid search method in terms of estimation root mean square error (RMSE) under different FDOA noise levels ($\sigma_f = 1, 2, 4, 8, 16\text{Hz}$).

Suppose in the l -th Monte Carlo run, the estimated source position in the ECEF coordinates is denoted by $\mathbf{u}_l$. The geolocation RMSE is computed using

$$\text{RMSE} = \sqrt{\frac{1}{L} \sum_{l=1}^L \|\mathbf{u}_l - \mathbf{u}^\circ\|^2}, \quad (5.9)$$

where L is the total number of Monte Carlo runs (normally set to be e.g, 10^4).

5.2.4 Guidelines for Final Report

1). Re-use your progress report that details the problem formulation and grid-based search method.

2). (10%) In order to generate the true FDOAs using (5.5), the signal wavelength needs to be provided. But it is still not known, as $\lambda = c/f_0$ requires the knowledge on the

unknown signal frequency f_0 . Find a way to obtain an estimate of the signal wavelength λ . Justify your design by carrying out an analysis on the approximation error.

Use this frequency estimate throughout the remaining experiments.

Bonus: In the current setup, we consider exploiting the FDOAs f_{21}, f_{31} and f_{41} only. The other possible FDOAs such as f_{23} and f_{34} are not used. Address the following question in your final report: Does utilizing these FDOAs bring any performance improvement? Justify your answer using theoretical analysis and you will receive 5% of your final report mark as bonus.

3). (30%) Derive with the help of chain rule the analytical expression of the gradient of the true FDOA $g(B^o, L^o, \mathbf{s}_j, \dot{\mathbf{s}}_j) - g(B^o, L^o, \mathbf{s}_1, \dot{\mathbf{s}}_1)$. Verify your result using the finite differences method for computing the gradient/derivatives (see [Chapter 4.4](#) of this note).

Bonus: if you can also numerically verify your results using the complex step method (see also [Chapter 4.4](#) of this note), you will receive 10% of your final report mark as bonus.

4). (35%) Select, modify and implement one of the following gradient-base methods **on your own**:

- Conjugate gradient
- Newton's method
- Gauss Newton method
- Levenberg-Marquardt algorithm
- BFGS algorithm

with line search and stopping criteria. See [Chapter 4.3](#) for an introduction to these gradient-based algorithms, line search and stopping criteria.

Make sure that in each iteration, the geolocation solution ALWAYS stays in the area of interest (i.e., the area with the latitude range $[0, 40^\circ]$ and longitude range $[0, 40^\circ]$).

In your implementation, also include a routine to realize the multistart strategy (i.e., generate multiple initial solutions such that multiple solution sequence can be produced using your gradient-based method. You can output the solution with the smallest objective function value as the final geolocation result). See [Chapter 4.3.6](#) for details.

5). (25%) Compare, using Monte Carlo simulation, the geolocation accuracy, in terms of estimation RMSE (see [\(5.9\)](#)), of your gradient-based algorithm against that of your grid search algorithm (now with the estimated signal frequency) under different frequency measurement noise levels.

Demonstrate the convergence behavior of your gradient-based algorithm in terms of the number of iterations required before convergence and the number of objective function evaluations needed in each line search.

Bonus: if you can implement another optimization algorithm, such as the particle swarm optimization (PSO), for performance comparison, you will receive 15% of your final report mark as bonus.

5.2.5 Guidelines for Demo

In your presentation, you need to cover at least item 1) from the progress report, as well as items 3), 4) and 5) from the final report.

In words, we expect to see your problem formulation, verification of gradient derivation, gradient-based algorithm design (i.e., the optimization strategy) and performance comparison with benchmark technique(s).

We encourage you to illustrate your design with various types of plots. As this project is in fact a 2-D optimization problem, visualization using contours (to show the modality of the objective function, satellite position, source position, etc), arrowed line segments to show the gradient direction and line search strategy, and RMSE curves a function of the FDOA measurement noise variance/standard deviation, can be used.

The marking scheme is similar to that of Project 1. We put emphasis on the presentation (20%) and graphic illustration (30%). The quality of your design (30%) and performance (20%) are also considered.

5.3 Suggested Project 3: Constraint-satisfaction

Brief highlights

- Come up with a problem, or pick one from the list below, and pose it in terms of a constraint-satisfaction scenario.
- Do a literature review to see what has been done on your problem.
- Define your constraints and identify them as either convex or non-convex.
- Derive projection operators and/or minimisation strategies to try and satisfy all constraints.
- You may use iterative projection algorithms, or any other algorithms you wish :)
- Implement your chosen solution strategy in code and test its effectiveness. Any programming language may be used²

5.3.1 Introduction

Many problems can be posed in terms of the satisfaction of various constraints. The solution to the problem requires all constraints be satisfied. The geometrical interpretation of this, is that the constraints are sets in some multi-dimensional space and the solution is located at the intersection of all the constraint sets (think Venn Diagrams). Many problems can naturally be posed as constraint satisfaction problems, including:

- Image reconstruction using X-ray diffraction (coherent diffractive imaging)
- Games and puzzles such as Sudoku
- Training neural networks
- Packing problems
- Project/resource allocation (think FYP and 150 students)
- Maximum-cut problems
- Boolean-SAT

5.3.2 Example solution method: Iterative Projection Algorithms

One way to arrive at an intersection of the constraints, and consequently solve the problem is via a class of algorithms called iterative projection algorithms (IPAs). Projection operators for individual constraint sets are constructed where they make the minimum possible change to the current estimate of the solution, referred to as the iterate $\mathbf{x}$, such that they satisfy the specific constraint - see the slides from the lectures for a picture and concrete examples.

The projection operators can be combined to form rules that update $\mathbf{x}$ in such a way that it becomes closer to satisfying all of the constraints. For a problem with two

²I hear Zig is good and Rust is the new Python

constraints, if we denote the projection operators onto these constraint sets by P_1 and P_2 , we can write

$$\mathbf{x}_{n+1} = \text{Rule}(\mathbf{x}_n, P_1, P_2) \quad (5.10)$$

where $\mathbf{x}_n$ is the iterate at the n th iteration of the algorithm. Applying such an update rule iteratively until $\mathbf{x}_n$ is no longer changing, i.e., reached a fixed-point, is the basis of iterative projection algorithms. Not all fixed-points are solutions however, and the update rule and its associated parameters must be chosen prudently.

The simplest IPA is the so-called error reduction (ER) algorithm (Fienup, 1982) where the n th iterate is updated according to the rule

$$\mathbf{x}_{n+1} = P_2 P_1 \mathbf{x}_n \quad (5.11)$$

The ER algorithm moves the iterate steadily towards a fixed-point but is unable to escape and explore other regions of the vector space should that fixed-point turn out not to be a solution. This is common when the constraints involved are non-convex in the multi-dimensional search space. A more effective update rule is the so-called Relaxed Reflect-Reflect (RRR) algorithm (Elser, 2017) given by

$$\mathbf{x}_{n+1} = \mathbf{x}_n - \beta \mathbf{x}_n + \beta R_2 R_1 \mathbf{x}_n \quad (5.12)$$

where β is a scalar parameter of the algorithm and

$$R_i \mathbf{x}_n = 2P_i \mathbf{x}_n - \mathbf{x}_n \quad (5.13)$$

is called the “reflection” operator, moving the iterate twice the distance into the constraint set labelled by i in the direction of the projection operation. The RRR algorithm is shown to have good convergence properties whilst having considerable ability to escape local minima, useful especially for combinatorially difficult problems where gradient methods perform poorly (Elser, Lan & Bendory, 2018), (Elser, Rankenburg, & Thibault, 2007).

More rules have been developed, many of which can be found in the paper by Marchesini (2007). Check out the articles in the reference Section 5.3.5 (also posted on Learn) and references therein for more in-depth discussions on IPAs and constraint-satisfaction problems.

5.3.3 Example Problem: Chrono-Coherent diffractive imaging

In X-ray coherent diffractive imaging (CDI), an object of interest is placed in the path of an X-ray beam. The diffracted X-rays are then measured on a detector. Under appropriate conditions, the measured signal is the *magnitude* of the Fourier transform of the object. The phase of the Fourier transform is lost since we are currently not able to directly measure the phases of electromagnetic waves at X-ray frequencies. Thankfully, the object can still be reconstructed if it is of a finite extent. The process of doing this is called phase retrieval, as we attempt to retrieve the Fourier phases by only knowing the Fourier magnitudes.

Posed as a constraint satisfaction problem, the two constraints that we have are (1) the measured Fourier magnitudes in Fourier space, and (2) the support of the object in real space.

The projection operators in phase retrieval are then P_S and P_M , which stands for the support and the magnitude projection operators, respectively. P_M makes the iterate

satisfy all constraints in Fourier space (the measured Fourier magnitude data) and is defined as

$$P_M \mathbf{x} = \mathcal{F}^{-1} \frac{M_{\text{data}}}{|\mathcal{F} \mathbf{x}|} \mathcal{F} \mathbf{x}, \quad (5.14)$$

where M_{data} is the measured Fourier magnitude and $\mathcal{F}$ is the Fourier transform operator, and $\mathcal{F}^{-1}$ the inverse Fourier transform operator. P_S makes the iterate satisfy all constraints in real space (finite support, positivity, etc.) which we can write simply as

$$P_S \mathbf{x} = \begin{cases} \mathbf{x} & \text{if inside support} \\ 0 & \text{if outside support} \end{cases} \quad (5.15)$$

The support projection zeros out all the values in the vector $\mathbf{x}$ that is outside the support of the object, and keeps the values within the support unchanged.

The update rule for the IPA is then formulated in such a way that the next iterate is generated from a combination of P_S , and P_M , and the current iterate at the n th iteration, $\mathbf{x}_n$. The IPA is iterated until $\mathbf{x}_{n+1} = \mathbf{x}_n$ to within numerical precision and the iterate is said to have reached a fixed-point.

The “Chrono” part in chrono-CDI refers to a time component of the problem. In CDI there are sometimes not enough information to allow a unique reconstruction from the intensity data (for example if the specimen is crystalline - see lecture slides for more details). So if many datasets are collected at different time points, the whole problem treated as a time-evolving system may become soluble. The different datasets need to be linked together with a third constraint, which we term a “temporal continuity constraint”. This constraint ensures that the retrieved solution varies smoothly from one time point to the next. Your task is to formulate this temporal continuity constraint and implement a projection operator that will enforce this constraint and connect together all of the datasets to see if the time-evolved system can be reconstructed.

This is an open-ended research problem and you are free to define and explore things in any way you wish!

5.3.4 Project brief and deliverables

- Feel free to choose a problem from the list in Section 5.3.1 or come up with your own.
- The progress report should cover the following:
 - Background / intuition about the problem;
 - An overview of possible and existing methods to use for the problem;
 - A plan for your solution;
 - Any progress-to-date.
- The final report should explain your solution in-depth, and be accompanied by the code used to implement your solution.
- You can upload the submissions to the Learn dropbox, and point me to your code on GitHub/Lab if you wish.

The final report will be assessed as in Table 5.3:

Item	Weight
Presentation	0.25
Quality of the technical design	0.50
Effort and Creativity	0.25

Table 5.3: Final report marking

See Jeremy's project for the definition of Presentation and Quality of technical design. Effort and creativity will be assessed through looking at your ideas, approach to the problem you've chosen, and the mathematics and figures you present in your reports.

I would recommend come and talk to Joe if you're interested in any of this stuff! :) as he doesn't have the time to type everything out here, and he can point you to a lot of other resources to help you understand the problems.

5.3.5 References

- Elser, V. (2003). The Mermin fixed point. *Foundations of Physics*, 33, 1691-1698.
- Elser, V., Rankenburg, I., & Thibault, P. (2007). Searching with iterated maps. *Proceedings of the National Academy of Sciences*, 104(2), 418-423.
- Elser, V. (2017). The complexity of bit retrieval. *IEEE Transactions on Information Theory*, 64(1), 412-428.
- Elser, V., Lan, T. Y., & Bendory, T. (2018). Benchmark problems for phase retrieval. *SIAM Journal on Imaging Sciences*, 11(4), 2429-2455.
- Fienup, J. R. (1982). Phase retrieval algorithms: a comparison. *Applied optics*, 21(15), 2758-2769.
- Gravel, S., & Elser, V. (2008). Divide and concur: A general approach to constraint satisfaction. *Physical Review E*, 78(3), 036706.
- Marchesini, S. (2007). A unified evaluation of iterative projection algorithms for phase retrieval. *Review of scientific instruments*, 78(1).
- Millane, R. P., & Lo, V. L. (2013). Iterative projection algorithms in protein crystallography. I. Theory. *Acta Crystallographica Section A: Foundations of Crystallography*, 69(5), 517-527.

Chapter 6

Mathematics Background: Part II

6.1 Gaussian Distribution

Perhaps, the most widely used probability distribution is the multivariate Gaussian, also called multivariate normal (MVN) in some literature. It is mathematically convenient, and more importantly, Gaussianity is fairly reasonable in many applications.

Suppose $\mathbf{x} \in \mathcal{R}^{n \times 1}$ follows a multivariate Gaussian distribution, i.e., $\mathbf{x} \sim \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}, \boldsymbol{\Sigma})$, which is defined as

$$\mathcal{N}(\mathbf{x}; \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{\sqrt{|2\pi\boldsymbol{\Sigma}|}} \exp\left(-\frac{(\mathbf{x} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu})}{2}\right). \quad (6.1)$$

Here, $\boldsymbol{\mu} = [\mu_1, \mu_2, \dots, \mu_n]^T = \mathbb{E}[\mathbf{x}]$ is the mean vector and $\boldsymbol{\Sigma} = \mathbb{E}[(\mathbf{x} - \boldsymbol{\mu})(\mathbf{x} - \boldsymbol{\mu})^T]$ is the covariance. The notation $|\cdot|$ represents the matrix determinant. In particular, the covariance matrix $\boldsymbol{\Sigma}$ is equal to

$$\boldsymbol{\Sigma} = \begin{bmatrix} \text{var}(x_1) & \text{cov}(x_1, x_2) & \cdots & \text{cov}(x_1, x_n) \\ \text{cov}(x_2, x_1) & \text{var}(x_2) & \cdots & \text{cov}(x_2, x_n) \\ \vdots & \vdots & \cdots & \vdots \\ \text{cov}(x_n, x_1) & \text{cov}(x_n, x_2) & \cdots & \text{var}(x_n) \end{bmatrix}, \quad (6.2)$$

where

$$\text{var}(x_i) = \mathbb{E}[(x_i - \mu_i)^2], \quad i = 1, 2, \dots, n, \quad (6.3)$$

and

$$\text{cov}(x_i, x_j) = \mathbb{E}[(x_i - \mu_i)(x_j - \mu_j)], \quad i, j = 1, 2, \dots, n, \text{ and } i \neq j. \quad (6.4)$$

It can be shown (Exercise 3.4) that $\boldsymbol{\Sigma}$ is symmetric and at least positive semidefinite. When $n = 1$, (6.1) reduces to the univariate Gaussian distribution (Exercise 3.5),

$$\mathcal{N}(x; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right). \quad (6.5)$$

One application of the Gaussian distribution is a transformation method popular in machine learning referred to as the re-parameterization trick. In some cases, the objective function is the expectation of another function $f(\mathbf{y})$ with respect to e.g., a Gaussian density $\mathcal{N}(\mathbf{y}; \boldsymbol{\mu}_y, \boldsymbol{\Sigma}_y)$, where the mean $\boldsymbol{\mu}_y$ and covariance $\boldsymbol{\Sigma}_y$ are the design variables to be optimized. The re-parameterization trick replaces $\mathbf{y}$ using

$$\mathbf{y} = \boldsymbol{\mu}_y + \mathbf{L}_y \boldsymbol{\epsilon}, \quad (6.6)$$

where $\Sigma_y = \mathbf{L}_y \mathbf{L}_y^T$ is the Cholesky decomposition of the covariance Σ_y , and ϵ follows a Gaussian density $\mathcal{N}(\epsilon; \mathbf{0}, \mathbf{I})$. As a result, the objective function becomes the expectation of $f(\mu_y + \mathbf{L}_y \epsilon)$ with respect to the Gaussian density $\mathcal{N}(\epsilon; \mathbf{0}, \mathbf{I})$ that is now independent of the design variables μ_y and Σ_y .

6.2 Maximum Likelihood Estimation

We are interested in estimating the parameter vector $\theta \in \mathcal{R}^{r \times 1}$ from measurements (also referred to as training data) in $\mathcal{D}$. The most commonly used approach for parameter estimation is to locate the parameter values that *assign* the highest probability to the measurements. This is called the maximum likelihood estimation (MLE), which indeed requires solving an optimization (maximization) problem. Mathematically, the MLE is defined as

$$\hat{\theta} = \max_{\theta} p(\mathcal{D}|\theta), \quad (6.7)$$

where $p(\mathcal{D}|\theta)$ is the distribution function of $\mathcal{D}$ parameterized by θ .

Sometimes, it is convenient to work with the logarithm of the objective function, which is $\log(p(\mathcal{D}|\theta))$. It can be shown (Exercise 3.6) that this transform does not change the solution to (6.7).

When the measurements in $\mathcal{D} = \{\mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_m\}$ are independent and identically distributed (i.i.d.), the logarithm of the original objective function becomes

$$\log(p(\mathcal{D}|\theta)) = \log \left(\sum_{i=1}^m p(\mathbf{y}_i|\theta) \right). \quad (6.8)$$

To illustrate the concept of MLE, consider the following example where the measurements are generated via

$$\mathbf{y} = [y_1, y_2, \dots, y_m]^T = \mathbf{G}\theta + \epsilon, \quad (6.9)$$

where $\mathbf{G}$ is a $m \times r$ measurement matrix. ϵ is the additive zero-mean Gaussian measurement noise (i.e., $\epsilon \sim \mathcal{N}(\epsilon; \mathbf{0}, \Sigma)$). It can be shown (Exercise 3.7) that the measurements in $\mathbf{y}$ follows a Gaussian distribution given by

$$p(\mathbf{y}|\theta) = \mathcal{N}(\mathbf{y}; \mathbf{G}\theta, \Sigma). \quad (6.10)$$

Note that in this case, the parameter vector θ determines the mean of the measurements, $\mathbf{G}\theta$, which according to (6.9), is also the unknown true values of the measurements.

To find the MLE of θ , we first notice that the logarithm of the objective function $p(\mathbf{y}|\theta)$ in (6.10) is, from (6.1),

$$\log(p(\mathbf{y}|\theta)) \propto -(\mathbf{y} - \mathbf{G}\theta)^T \Sigma^{-1} (\mathbf{y} - \mathbf{G}\theta), \quad (6.11)$$

where all the terms not depending on θ have been dropped. The MLE of θ for this particular example is thus given by

$$\hat{\theta} = \max_{\theta} \log(p(\mathbf{y}|\theta)) = \min_{\theta} (\mathbf{y} - \mathbf{G}\theta)^T \Sigma^{-1} (\mathbf{y} - \mathbf{G}\theta). \quad (6.12)$$

If the covariance of the noise ϵ , Σ , is a diagonal matrix with the diagonal elements being equal to one another (i.e., $\Sigma = \sigma^2 \mathbf{I}$), the MLE problem in (6.12) reduces to

$$\hat{\theta} = \max_{\theta} \log(p(\mathbf{y}|\theta)) = \min_{\theta} (\mathbf{y} - \mathbf{G}\theta)^T (\mathbf{y} - \mathbf{G}\theta). \quad (6.13)$$

This is in fact a linear least squares (LLS) problem. If Σ has non-identical diagonal elements and/or it is not a diagonal matrix, the MLE problem in (6.12) is a weighted least squares (WLS) problem in the sense that each measurement is not equally weighted in the objective function. Ideally, we can emphasize those measurements if we know they have better quality, to improve the estimation accuracy. It can be shown by using the results in Chapter 2.3 of the course notes (Exercise 3.8) that the MLE from (6.12) is

$$\hat{\boldsymbol{\theta}} = (\mathbf{G}^T \Sigma^{-1} \mathbf{G})^{-1} \mathbf{G}^T \Sigma^{-1} \mathbf{y}. \quad (6.14)$$

An important extension to the measurement model in (6.9) is that the true measurements are now *nonlinearly* related to the unknown parameters. Mathematically, we have

$$\mathbf{y} = \mathbf{g}(\boldsymbol{\theta}) + \boldsymbol{\epsilon}. \quad (6.15)$$

Here, the nonlinear measurement function $\mathbf{g}(\boldsymbol{\theta})$ is defined as

$$\mathbf{g}(\boldsymbol{\theta}) = [g_1(\boldsymbol{\theta}), g_2(\boldsymbol{\theta}), \dots, g_m(\boldsymbol{\theta})]^T, \quad (6.16)$$

where $g_i(\boldsymbol{\theta})$ produces the true value of the i -th measurement y_i , $i = 1, 2, \dots, m$.

In this case, the MLE of $\boldsymbol{\theta}$ is found by solving

$$\hat{\boldsymbol{\theta}} = \max_{\boldsymbol{\theta}} \log(p(\mathbf{y}|\boldsymbol{\theta})) = \min_{\boldsymbol{\theta}} (\mathbf{y} - \mathbf{g}(\boldsymbol{\theta}))^T \Sigma^{-1} (\mathbf{y} - \mathbf{g}(\boldsymbol{\theta})). \quad (6.17)$$

This is called the nonlinear least squares (NLS) problem. This formulation will be explored in the course project provide by Le Yang, it is an unconstrained optimization problem that can be solved by methods that was presented in Chapter 3 of this notes.

Chapter 7

Sparsity-aware Optimization

7.1 Motivation Examples

7.2 Majorization-Minimization Algorithm

Chapter 8

Appendices

© 2013 Thomas Schneider

Released under [CC-BY-SA](#). If you want to reuse it, please respect the attribution clause by linking to the [original document](#) ([emailing me](#) would also be nice).

Many Thanks to:

- James Heisig for inspiring this template with his book Remembering the Kanji.

Smart Kanji Book Template

Inspired by Remembering the Kanji

漢字 : keyword

parts, parts

Story

word definition